Báo cáo P1 — Các yếu tố ảnh hưởng đến điểm cuối kỳ (UCI Student Performance, `mat`)

> Báo cáo được sinh tự động bởi `report_builder.py` ngày 2026-10-13, từ cùng kết quả được ghi vào `data/processed/summary.json`. Mọi con số do pipeline tính ra, mọi đoạn code được đọc trực tiếp từ file nguồn hiện tại, nên báo cáo không thể lệch với code hay dữ liệu. Tạo lại bằng `python main.py`.

Câu hỏi chính: Thời gian tự học, số buổi vắng, hỗ trợ từ gia đình và điểm giữa kỳ (G1, G2) liên quan thế nào đến điểm cuối kỳ (G3)?

1. Tóm tắt

Câu hỏi	Trả lời (môn Toán, 357 học sinh có G3 > 0, trừ khi ghi khác)
Điểm giữa kỳ → G3	Liên hệ mạnh nhất: r(G2, G3) = **0.966**, r(G1, G3) = 0.892
Số lần trượt trước → G3	0 lần: **11.94**, 1: 10.15, 2: 8.83, 3: 8.27 (r = -0.294)
Vắng học → G3	thấp 12.26 / trung bình 11.38 / cao 10.37 (r = **-0.213**, ANOVA p Holm < 0.001)
Thời gian tự học → G3	dương nhưng yếu, r = 0.127 (ANOVA p Holm 0.019, eta^2 = 0.036: rất nhỏ)
Internet ở nhà → G3	có 11.68 vs không 10.71; p gốc 0.027, p Holm 0.080 → **không còn ý nghĩa sau hiệu chỉnh**
Xét tất cả yếu tố cùng lúc (hồi quy)	các yếu tố nền giải thích R^2 = 0.234; có ý nghĩa ở cả 2 môn khi giữ các yếu tố khác cố định: **studytime (+1 mức), failures (+1 lần trượt), absences (+1 buổi vắng), schoolsup (có vs không)**; internet sau điều chỉnh b = +0.53 (p 0.247)
Hỗ trợ gia đình (`famsup`) → G3	có 11.35 vs không 11.8 (p Holm 0.404): **không có khác biệt có ý nghĩa**
Tỉ lệ đỗ	67.1% trên toàn bộ 395 học sinh; 74.2% sau khi bỏ 38 em nghỉ bỏ thi
Rule cảnh báo sớm (không dùng G3)	đặt trên mat: recall 93.8%, precision 81.9%; **kiểm tra trên học sinh môn Bồ Đào Nha chưa từng thấy: recall 91.4%, precision 66.0%**

Phát hiện quan trọng về chất lượng dữ liệu: 38 học sinh có G3 = 0 và **38 em trong số đó có 0 buổi vắng được ghi nhận**. Nhóm này được gắn cờ *dropout suspect* (nghỉ bỏ thi), không xoá. Nếu để lẫn vào, tương quan vắng học–G3 thành r = 0.034 và che mất ảnh hưởng tiêu cực thật sự của vắng học.

2. Đối chiếu yêu cầu đề bài

Yêu cầu (bảng đề)	Thực hiện ở đâu	Mục
Dataset UCI Student Performance (`student.zip`)	`fetch_uci_raw()`	3.1
Web scraping Wikipedia PISA	`scrape_pisa_table()`, `_parse_pisa_html()`, `validate_pisa_raw()`	3.2
Lưu raw CSV	`data/raw/student-*.csv`, `data/raw/pisa_reference.csv`	3
Làm sạch kiểu dữ liệu, encode yes/no	`clean_data()`	5

Yêu cầu (bảng đề)	Thực hiện ở đâu	Mục
4 hàm cơ bản (yes/no, pass/fail, phân loại vắng, điểm trung bình)	`convert_yes_no`, `calc_pass_fail`, `classify_absences`, `calc_average_grade`	4
list of dicts, dict mapping, set trường/nghề, tuple ngưỡng	`SAMPLE_STUDENTS`, `VAR_DESCRIPTIONS`, `set()` trong `analyze()`, `THRESHOLDS`	4
EDA bằng groupby, mean, std	`_grp()`, `_scope_stats()`	7
So sánh G3 theo studytime, absences, failures, internet	các bảng `by_*`	7
Tính pass/fail; phân nhóm rủi ro học tập	`pass_fail`, `classify_risk()`, `_risk_eval()`	7, 9
Tương quan G1, G2, G3	`corr_G1_G2_G3`, heatmap	7, 8.3
Vẽ bar, histogram, heatmap	`_plot_bar`, `_plot_hist`, `_plot_heatmap`	8
OOP: `load_data()`, `clean_data()`, `analyze()`, `export_results()`; `main.py` xuất `cleaned_students.csv` + `summary.json`	`StudentDataProcessor`, `main.py`	11
Insight và giới hạn dữ liệu	—	12, 13

3. Thu thập dữ liệu

3.1 UCI Student Performance

Đã làm gì: chỉ khi chưa có file raw thì mới tải file zip chính thức của UCI, rồi giải nén `student.zip` bên trong vào `data/raw/`. File raw không bao giờ bị sửa sau đó.

File	Số dòng	Số cột	Vai trò
`student-mat.csv`	395	33	Phân tích chính (môn Toán)
`student-por.csv`	649	33	Đối chứng / so sánh (môn Bồ Đào Nha)

`data_processor.py` — `fetch_uci_raw` (lines 165–178)

```python
def fetch_uci_raw(raw_dir: Path, url: str = UCI_ZIP_URL) -> list[Path]:
    """Download UCI student.zip and extract raw files if any are missing."""
    raw_dir = Path(raw_dir)
    targets = [raw_dir / name for name in UCI_FILES]
    if all(p.exists() for p in targets):
        return targets
    raw_dir.mkdir(parents=True, exist_ok=True)
    resp = requests.get(url, timeout=60, headers=HTTP_HEADERS)
    resp.raise_for_status()
    with zipfile.ZipFile(io.BytesIO(resp.content)) as outer:
        with zipfile.ZipFile(io.BytesIO(outer.read("student.zip"))) as inner:
            for name in UCI_FILES:
                (raw_dir / name).write_bytes(inner.read(name))
    return targets
```

Vì sao tin được: URL là link tải chính thức của UCI; file giữ nguyên như UCI phát hành; số dòng/cột khớp mô tả của UCI (395 / 649 học sinh, 33 thuộc tính).

3.2 Cào dữ liệu PISA từ Wikipedia

Đã làm gì:

1. `requests.get` tải HTML của bài viết PISA (không cần trình duyệt vì bảng nằm sẵn trong HTML).
2. `BeautifulSoup` tìm mọi `table.wikitable` và giữ 2 loại bảng:
   - **summary**: bảng xếp hạng mới nhất (`Rank | Country/economy | Score`), mỗi môn 1 bảng con;
   - **comparison**: "Rankings comparison 2000–2022", mỗi nước 1 dòng với cặp `score, rank` theo từng năm (chuyển từ dạng rộng sang dạng dài).
3. Kết quả (2342 dòng, giữ nguyên chuỗi gốc của trang) lưu vào `data/raw/pisa_reference.csv`.
4. Cơ chế **cache-first**: chạy bình thường thì đọc file đã lưu; muốn cào lại thì chạy `python main.py --refresh-pisa`.

Nguồn gốc	Giá trị
Phiên bản (revision) Wikipedia	1378067005
Thời điểm cào (UTC)	2026-10-13T06:44:49+00:00
Số dòng thô	2342
Giấy phép	CC BY-SA 4.0 (Wikipedia)

`data_processor.py` — `scrape_pisa_table` (lines 301–339)

```
def scrape_pisa_table(url: str = PISA_URL, out_csv: Path | None = None,
                      refresh: bool = False) -> pd.DataFrame:
    """Cache-first PISA scrape from Wikipedia into a raw long CSV.

    - Cache exists and refresh=False: read and validate the cache, no network.
    - Otherwise fetch live, parse, validate; only a valid result overwrites the
      cache, together with <out_csv>.meta.json (url, revision id, permalink, time).
    - Live fetch/parse/validation fails: warn and fall back to a valid cache,
      else raise RuntimeError.
    """
    cache = Path(out_csv) if out_csv is not None else None
    if cache is not None and cache.exists() and not refresh:
        return _read_pisa_cache(cache)
    try:
        resp = requests.get(url, timeout=25, headers=HTTP_HEADERS)
        resp.raise_for_status()
        resp.encoding = "utf-8"
        df = _parse_pisa_html(resp.text)
        validate_pisa_raw(df)
    except Exception as exc:
        if cache is not None and cache.exists():
            warnings.warn(f"Live PISA scrape failed ({type(exc).__name__}: {exc}); "
                          f"keeping cached {cache.name}", stacklevel=2)
            return _read_pisa_cache(cache)
        raise RuntimeError(f"PISA scrape failed: {type(exc).__name__}: {exc}") from exc
    if cache is not None:
        cache.parent.mkdir(parents=True, exist_ok=True)
        df.to_csv(cache, index=False, encoding="utf-8")
        rev = re.search(r'"wgRevisionId":(\d+)', resp.text)
        meta = {
            "url": url,
            "revision_id": int(rev.group(1)) if rev else None,
            "permalink": (f"https://en.wikipedia.org/w/index.php?oldid={rev.group(1)}"
                          if rev else None),
            "scraped_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
            "n_rows": int(len(df)),
        }
        _meta_path(cache).write_text(json.dumps(meta, indent=2), encoding="utf-8")
    return df
```

`data_processor.py` — `_parse_pisa_html` (lines 189–243)

```
def _parse_pisa_html(html: str) -> pd.DataFrame:
    """Extract PISA ranking tables into a raw long table (all values as page strings).

    - summary  : latest 'ranking summary' tables (Rank | Country/economy | Score),
```

```
                subject order taken from the container table header.
    - comparison: 'Rankings comparison' tables (Country | year score/rank pairs).
    """
    soup = BeautifulSoup(html, "html.parser")
    rows: list[dict] = []
    for table in soup.find_all("table", class_="wikitable"):
        trs = table.find_all("tr", recursive=False) or table.select(":scope > tbody > tr")
        if not trs:
            continue
        head = _cell_texts(trs[0])
        heading = table.find_previous(["h2", "h3"])
        heading_text = heading.get_text(" ", strip=True) if heading else ""

        # Container of the latest ranking summary: header lists the subjects,
        # nested tables hold Rank | Country/economy | Score in the same order.
        subjects = [_strip_footnote(h).lower() for h in head]
        nested = table.find_all("table", class_="wikitable")
        if nested and all(s in PISA_SUBJECTS for s in subjects) and len(nested) == len(subjects):
            year_match = re.search(r"(\d{4})", heading_text)
            year = year_match.group(1) if year_match else ""
            for subject, sub in zip(subjects, nested):
                for tr in sub.find_all("tr")[1:]:
                    cells = _cell_texts(tr)
                    if len(cells) == 3:
                        rank, country, score = cells
                    elif len(cells) == 2:  # e.g. "International Average (OECD)" has no rank
                        rank, (country, score) = "", cells
                    else:
                        continue
                    rows.append({"source": "summary", "year": year,
                                 "subject": PISA_SUBJECTS[subject],
                                 "rank": rank, "country": country, "score": score})
            continue

        # Historical comparison: single-cell subject title, then Country | years, then Score/Rank.
        if (len(head) == 1 and _strip_footnote(head[0]).lower() in PISA_SUBJECTS
                and len(trs) > 3 and _cell_texts(trs[1])[:1] == ["Country"]):
            subject = PISA_SUBJECTS[_strip_footnote(head[0]).lower()]
            years = [_strip_footnote(y) for y in _cell_texts(trs[1])[1:]]
            for tr in trs[3:]:
                cells = _cell_texts(tr)
                if len(cells) != 1 + 2 * len(years):
                    continue
                country = cells[0]
                for k, year in enumerate(years):
                    rows.append({"source": "comparison", "year": year, "subject": subject,
                                 "rank": cells[2 + 2 * k], "country": country,
                                 "score": cells[1 + 2 * k]})
    if not rows:
        raise ValueError("No PISA ranking table found")
    return pd.DataFrame(rows, columns=["source", "year", "subject", "rank", "country", "score"])
```

Vì sao tin được — kiểm tra cấu trúc. Mọi bảng cào về (và mọi lần đọc cache) phải qua `validate_pisa_raw()`; không đạt thì bị loại và giữ nguyên cache cũ:

- cả 2 loại bảng đều có đủ 3 môn;
- năm là số trong khoảng 2000 đến năm hiện tại;
- mọi điểm nằm trong [200, 700];
- mỗi (nguồn, môn, năm) có ≥ 30 nước có điểm, hạng là số, không trùng nước và có đúng 1 dòng trung bình OECD.

`data_processor.py` — `validate_pisa_raw` (lines 250–282)

```
def validate_pisa_raw(df: pd.DataFrame) -> None:
    """Sanity-check a raw PISA table; raise ValueError listing every problem.

    Catches silent mis-parses after a Wikipedia layout change: missing table
    families, wrong subject/year labels, empty or implausible score columns.
    """
```

```
    problems: list[str] = []
    for src in ("summary", "comparison"):
        got = set(df.loc[df["source"] == src, "subject"])
        if got != set(PISA_SUBJECTS.values()):
            problems.append(f"{src}: subjects {sorted(got)} != {sorted(PISA_SUBJECTS.values())}")
    years = _to_num(df["year"])
    bad_year = years.isna() | ~years.between(2000, datetime.now().year)
    if bad_year.any():
        problems.append(f"invalid year values: {sorted(set(df.loc[bad_year, 'year']))}")
    score = _to_num(df["score"])
    out_of_range = score.notna() & ~score.between(PISA_SCORE_MIN, PISA_SCORE_MAX)
    if out_of_range.any():
        problems.append(f"{int(out_of_range.sum())} scores outside "
                        f"[{PISA_SCORE_MIN}, {PISA_SCORE_MAX}]")
    is_avg = df["country"].str.contains("International Average", case=False)
    for (src, subj, year), g in df.groupby(["source", "subject", "year"]):
        countries = g[~is_avg.loc[g.index] & score.loc[g.index].notna()]
        if len(countries) < PISA_MIN_COUNTRIES:
            problems.append(f"{src}/{subj}/{year}: only {len(countries)} countries with a score")
        if _to_num(countries["rank"]).isna().any():
            problems.append(f"{src}/{subj}/{year}: country rows without numeric rank")
        if countries["country"].duplicated().any():
            problems.append(f"{src}/{subj}/{year}: duplicated countries")
        if int(is_avg.loc[g.index].sum()) != 1:
            problems.append(f"{src}/{subj}/{year}: expected exactly 1 OECD average row")
    if problems:
        raise ValueError("PISA validation failed: " + "; ".join(problems))
```

3.3 Làm sạch PISA

Bỏ chú thích (`[41]`) và dấu `*` cuối tên nước, `–` (không tham gia) thành giá trị thiếu, rank/score/year đổi sang số, "International Average (OECD)" đổi tên thành `OECD average`. Kết quả: 1662 dòng trong `data/processed/pisa_clean.csv`.

`data_processor.py` — `clean_pisa` (lines 342–358)

```
def clean_pisa(raw: pd.DataFrame) -> pd.DataFrame:
    """Normalise the raw PISA long table: numeric rank/score/year, clean country names."""
    df = raw.copy()
    df["country"] = (df["country"].astype(str)
                     .map(_strip_footnote)
                     .str.replace(r"\s*\*+$", "", regex=True)
                     .str.strip())
    df.loc[df["country"].str.contains("International Average", case=False), "country"] = "OECD average"
    for col in ["year", "rank", "score"]:
        df[col] = _to_num(df[col])
    df = df.dropna(subset=["year", "score"])  # country did not take part that year
    df["year"] = df["year"].astype(int)
    df["score"] = df["score"].astype(int)
    df["rank"] = df["rank"].astype("Int64")
    return (df.drop_duplicates(subset=["year", "subject", "country"])
              .sort_values(["subject", "year", "rank"])
              .reset_index(drop=True))
```

3.4 Đối chiếu với báo cáo chính thức của OECD

Đã làm gì. Wikipedia là nguồn thứ cấp, nên số liệu cào về được so với **ấn phẩm chính thức của OECD**: OECD (2026), PISA 2025 Results (Volume I): Future-Ready Students, OECD Publishing, Paris, Table I.1. Snapshot of performance in science, reading, mathematics and computational problem-solving (page 30) (DOI, giấy phép CC BY 4.0). Bảng này được trích một lần từ file PDF của OECD vào `data/raw/oecd_pisa2025_table_I1.csv` (92 dòng: trung bình OECD + mọi quốc gia tham gia). Có 5 tên nước khác nhau giữa 2 nguồn và đã được ánh xạ (`OECD_NAME_ALIASES`, vd "Chinese Taipei" → "Taiwan").

Kỳ thi	Cách so	Số cặp so	Trong ngưỡng	Khớp tuyệt đối	Lệch lớn nhất	Không khớp (Wikipedia vs OECD)
2025	so trực tiếp, phải khớp tuyệt đối	274	274	274	0	—
2022	suy ra = điểm 2025 − mức thay đổi, cho lệch ±1 do làm tròn	211	208	161	7	Palestinian Authority mathematics 366 vs 372; Palestinian Authority science 369 vs 376; Palestinian Authority reading 349 vs 356

Nước có trong bảng OECD nhưng thiếu trong bảng Wikipedia: mathematics: Uzbekistan; reading: Uzbekistan.

Bồ Đào Nha và trung bình OECD (các dòng báo cáo này thực sự dùng):

Quốc gia	Môn	Wikipedia	OECD chính thức	Khớp
Portugal	mathematics	460	460	✅
Portugal	reading	462	462	✅
Portugal	science	482	482	✅
OECD average	mathematics	463	463	✅
OECD average	reading	461	461	✅
OECD average	science	482	482	✅

Vì sao tin được: mọi điểm 2025 cào từ Wikipedia đều trùng với giá trị chính thức của OECD; điểm 2022 (suy ra từ cột mức thay đổi của OECD) khớp trong phạm vi làm tròn, trừ các dòng ghi ở cột không khớp, và các dòng đó không ảnh hưởng đến Bồ Đào Nha. Các kỳ cũ hơn (2000–2018) không có trong bảng OECD này nên vẫn chỉ dựa vào Wikipedia.

`data_processor.py` — `cross_check_pisa` (lines 361–402)

```
def cross_check_pisa(pisa: pd.DataFrame, oecd: pd.DataFrame) -> dict:
    """Compare scraped Wikipedia PISA scores with the official OECD Table I.1.

    - OECD_PISA_YEAR scores must match exactly (both are rounded means).
    - The previous cycle (2022) is derived as score - change; because OECD
      computes the change from unrounded means, a difference of 1 is allowed.
    """
    o = oecd.copy()
    o["country"] = o["country"].str.replace(r"\s*\*+$", "", regex=True).str.strip().replace(OECD_NAME_ALIASES)
    rows = []
    for subject in PISA_SUBJECTS.values():
        score, change = _to_num(o[subject]), _to_num(o[f"{subject}_change"])
        rows.append(pd.DataFrame({"country": o["country"], "subject": subject,
                                  "year": OECD_PISA_YEAR, "oecd": score}))
        rows.append(pd.DataFrame({"country": o["country"], "subject": subject,
                                  "year": OECD_PISA_YEAR - 3, "oecd": score - change}))
    ref = pd.concat(rows).dropna(subset=["oecd"])
    merged = ref.merge(pisa[["country", "subject", "year", "score"]],
                       on=["country", "subject", "year"], how="inner")
    merged["diff"] = merged["score"] - merged["oecd"]
    out: dict = {}
    for year, tol in ((OECD_PISA_YEAR, 0), (OECD_PISA_YEAR - 3, 1)):
        g = merged[merged["year"] == year]
        bad = g[g["diff"].abs() > tol]
        out[str(year)] = {
            "tolerance": tol,
            "n_compared": int(len(g)),
            "n_within_tolerance": int(len(g) - len(bad)),
            "n_exact": int((g["diff"] == 0).sum()),
            "max_abs_diff": int(g["diff"].abs().max()) if len(g) else None,
            "mismatches": bad[["country", "subject", "score", "oecd"]].astype(
```

```
                {"score": int, "oecd": int}).to_dict("records"),
        }
    latest = pisa[pisa["year"] == OECD_PISA_YEAR]
    official = set(o["country"])
    out["missing_in_wikipedia"] = {
        s: sorted(official - set(latest.loc[latest["subject"] == s, "country"]))
        for s in PISA_SUBJECTS.values()}
    out["missing_in_oecd"] = {
        s: sorted(set(latest.loc[latest["subject"] == s, "country"]) - official)
        for s in PISA_SUBJECTS.values()}
    return out
```

4. Python cơ bản theo yêu cầu đề

Cấu hình (tuple, dict mapping, list of dicts) — code thật:

`data_processor.py` (lines 27–101)

```
# Threshold config as tuples (immutable, per rubric)
# ------------------------------------------------------------------------
# (PASS_MARK, G_WARN, ABS_MED, ABS_HIGH, FAIL_HIGH)
THRESHOLDS = (10, 10, 4, 10, 2)
PASS_MARK, G_WARN, ABS_MED, ABS_HIGH, FAIL_HIGH = THRESHOLDS

# Anomaly flags: (IQR_K, G1_DROP, G2_DROP, AGE_OUTLIER)
ANOMALY_THRESHOLDS = (1.5, 8, 6, 20)
IQR_K, G1_DROP, G2_DROP, AGE_OUTLIER = ANOMALY_THRESHOLDS

# PISA scrape sanity checks: (MIN_COUNTRIES per subject/year, SCORE_MIN, SCORE_MAX)
PISA_CHECKS = (30, 200, 700)
PISA_MIN_COUNTRIES, PISA_SCORE_MIN, PISA_SCORE_MAX = PISA_CHECKS

# Statistical tests: significance level and the variables whose G3 groups are compared
ALPHA = 0.05
BINARY_TEST_COLS = ["internet", "famsup", "schoolsup", "paid"]
MULTI_TEST_COLS = ["studytime", "absence_level", "failures"]
CORR_TEST_COLS = ["G1", "G2", "studytime", "failures", "absences", "famsup_bin"]

# Columns identifying the same student in student-mat and student-por (student-merge.R)
MERGE_KEYS = ["school", "sex", "age", "address", "famsize", "Pstatus", "Medu", "Fedu",
              "Mjob", "Fjob", "reason", "nursery", "internet"]

# Multivariable regression of G3 (each coefficient = effect with all other columns held fixed).
# Background factors: the main-question variables plus socio-economic / demographic controls.
REG_FACTORS = ["studytime", "failures", "absences", "famsup_bin", "schoolsup_bin", "paid_bin",
               "internet_bin", "higher_bin", "Medu", "Fedu", "male", "age", "urban"]
REG_GRADES = ["G1", "G2"]

# OECD PISA 2025 Table I.1 (official source used to cross-check the Wikipedia scrape).
# Country names that differ between the OECD report and Wikipedia:
OECD_PISA_YEAR = 2025
OECD_NAME_ALIASES = {"Chinese Taipei": "Taiwan", "Czechia": "Czech Republic", "Korea": "South Korea",
                     "Slovak Republic": "Slovakia", "Viet Nam": "Vietnam"}

# Variable description mapping (dict mapping, per rubric)
VAR_DESCRIPTIONS = {
    "studytime": "Weekly study time bucket 1-4 (<2h, 2-5h, 5-10h, >10h)",
    "absences": "Number of school absences",
    "failures": "Number of past class failures",
    "internet": "Internet access at home (yes/no)",
    "G1": "First period grade (0-20)",
    "G2": "Second period grade (0-20)",
    "G3": "Final grade (0-20)",
    "schoolsup": "Extra educational support from school (yes/no)",
    "famsup": "Family educational support (yes/no)",
    "paid": "Extra paid classes within the course subject (yes/no)",
}

# Valid value domain per numeric column (from data/raw/student.txt)
VALID_RANGES = {
```

```
    "age": (15, 22), "Medu": (0, 4), "Fedu": (0, 4),
    "traveltime": (1, 4), "studytime": (1, 4), "failures": (0, 3),
    "famrel": (1, 5), "freetime": (1, 5), "goout": (1, 5),
    "Dalc": (1, 5), "Walc": (1, 5), "health": (1, 5),
    "absences": (0, 93), "G1": (0, 20), "G2": (0, 20), "G3": (0, 20),
}
REQUIRED_COLS = ["G1", "G2", "G3", "absences", "failures", "studytime", "age"]

YES_NO_COLS = ["schoolsup", "famsup", "paid", "activities", "nursery",
               "higher", "internet", "romantic"]

FLAG_COLS = ["dropout_suspect", "g2_zero", "grade_drop",
             "absence_outlier", "age_outlier", "inconsistent_grade"]

# Sample students as list of dicts (per rubric)
SAMPLE_STUDENTS = [
    {"id": 1, "studytime": 2, "absences": 6, "failures": 0,
     "internet": "yes", "G1": 5, "G2": 6, "G3": 6},
    {"id": 2, "studytime": 2, "absences": 4, "failures": 0,
     "internet": "no", "G1": 5, "G2": 5, "G3": 6},
    {"id": 3, "studytime": 3, "absences": 10, "failures": 3,
     "internet": "yes", "G1": 7, "G2": 8, "G3": 10},
]
```

Bốn hàm cơ bản:

`data_processor.py` — `convert_yes_no` (lines 107–114)

```
def convert_yes_no(value) -> int | None:
    """Encode 'yes'/'no' (case-insensitive, stripped) to 1/0; anything else -> None."""
    text = str(value).strip().lower()
    if text == "yes":
        return 1
    if text == "no":
        return 0
    return None
```

`data_processor.py` — `calc_pass_fail` (lines 117–119)

```
def calc_pass_fail(g3: float, pass_mark: int = PASS_MARK) -> str:
    """Return 'pass' if G3 >= pass_mark else 'fail'."""
    return "pass" if float(g3) >= pass_mark else "fail"
```

`data_processor.py` — `classify_absences` (lines 122–129)

```
def classify_absences(n: int) -> str:
    """Bucket absences: low (0-3) / medium (4-9) / high (>=10)."""
    n = int(n)
    if n < ABS_MED:
        return "low"
    if n < ABS_HIGH:
        return "medium"
    return "high"
```

`data_processor.py` — `calc_average_grade` (lines 132–134)

```
def calc_average_grade(g1: float, g2: float, g3: float) -> float:
    """Mean of the three grade periods (descriptive only: contains G3)."""
    return (float(g1) + float(g2) + float(g3)) / 3.0
```

`set()` các giá trị duy nhất (tính trong `analyze()`): trường = `['GP', 'MS']`, nghề của mẹ = `['at_home', 'health', 'other', 'services', 'teacher']`, nghề của bố = `['at_home', 'health', 'other', 'services', 'teacher']`.

5. Làm sạch dữ liệu

Đã làm gì (`StudentDataProcessor.clean_data()`, theo thứ tự: ép kiểu → kiểm tra → loại bỏ → tạo cột → gắn cờ). Số dòng vào: 1044 (cả 2 môn) → số dòng ra: 1044.

Bước	Cách làm	Số dòng/ô bị ảnh hưởng
1. Cắt khoảng trắng ở các cột chữ	`str.strip()`	0
2a. Ép kiểu số 16 cột số	`pd.to_numeric(errors='coerce')`	0
2b. Kiểm tra miền giá trị theo `student.txt`	`VALID_RANGES` → ngoài miền thành NaN	0
3. Mã hoá yes/no → 1/0, giá trị lạ → NaN	`convert_yes_no`	0
4a. Xoá dòng trùng	`drop_duplicates()`	0
4b. Xoá dòng thiếu cột bắt buộc	`dropna(subset=REQUIRED_COLS)`	0

Cột tạo thêm: `pass_fail` (G3 ≥ 10), `absence_level` (thấp 0–3 / trung bình 4–9 / cao ≥ 10), `avg_grade` (G1+G2+G3)/3 (chỉ để mô tả — có chứa G3), `avg_g1g2`, `risk_group`, và 6 cột cờ bất thường.

`data_processor.py` — `StudentDataProcessor.load_data` (lines 617–628)

```python
def load_data(self) -> pd.DataFrame:
    frames = []
    for subject, path in self.raw_paths.items():
        part = pd.read_csv(path, sep=";", dtype=str, keep_default_na=False)
        part.insert(0, "subject", subject)
        frames.append(part)
    self.df = pd.concat(frames, ignore_index=True)
    if self.pisa_raw is not None and self.pisa_raw.exists():
        self.pisa_df = pd.read_csv(self.pisa_raw, dtype=str, keep_default_na=False)
    if self.oecd_raw is not None and self.oecd_raw.exists():
        self.oecd_df = pd.read_csv(self.oecd_raw, dtype=str, keep_default_na=False)
    return self.df
```

`data_processor.py` — `StudentDataProcessor.clean_data` (lines 631–694)

```python
def clean_data(self) -> pd.DataFrame:
    assert self.df is not None, "call load_data() first"
    df = self.df.copy()
    log: dict = {"rows_in": int(len(df))}

    # 1) whitespace
    str_cols = [c for c in df.columns if c not in VALID_RANGES]
    stripped = df[str_cols].apply(lambda s: s.str.strip())
    log["n_cells_stripped"] = int((stripped != df[str_cols]).sum().sum())
    df[str_cols] = stripped

    # 2) numeric casts + domain checks (bad values -> NaN)
    log["n_non_numeric"], log["n_out_of_range"] = {}, {}
    for col, (lo, hi) in VALID_RANGES.items():
        num = pd.to_numeric(df[col], errors="coerce")
        log["n_non_numeric"][col] = int((num.isna() & (df[col] != "")).sum())
        bad = num.notna() & ~num.between(lo, hi)
        log["n_out_of_range"][col] = int(bad.sum())
        df[col] = num.mask(bad)

    # 3) yes/no encoding with validation
    log["n_invalid_yes_no"] = {}
    for col in YES_NO_COLS:
        enc = df[col].map(convert_yes_no)
        log["n_invalid_yes_no"][col] = int(enc.isna().sum())
        df[col + "_bin"] = enc.astype("Int64")

    # 4) duplicates (within subject) + missing required values
    before = len(df)
```

```
    df = df.drop_duplicates()
    log["n_duplicates_dropped"] = int(before - len(df))
    before = len(df)
    df = df.dropna(subset=REQUIRED_COLS).reset_index(drop=True)
    log["n_missing_required_dropped"] = int(before - len(df))
    for col in VALID_RANGES:
        if df[col].notna().all():
            df[col] = df[col].astype(int)

    # 5) derived columns
    df["pass_fail"] = df["G3"].apply(calc_pass_fail)
    df["absence_level"] = df["absences"].apply(classify_absences)
    df["avg_grade"] = df.apply(
        lambda r: calc_average_grade(r["G1"], r["G2"], r["G3"]), axis=1).round(2)
    df["avg_g1g2"] = ((df["G1"] + df["G2"]) / 2).round(2)
    df["risk_group"] = df.apply(classify_risk, axis=1)

    # 6) anomaly flags (flag, never delete); absence fence per subject
    q = df.groupby("subject")["absences"].quantile([0.25, 0.75]).unstack()
    fences = (q[0.75] + IQR_K * (q[0.75] - q[0.25])).to_dict()
    df["dropout_suspect"] = df["G3"] == 0
    df["g2_zero"] = df["G2"] == 0
    df["grade_drop"] = (((df["G1"] - df["G3"]) >= G1_DROP) | ((df["G2"] - df["G3"]) >= G2_DROP)) \
        & ~df["dropout_suspect"]
    df["absence_outlier"] = df["absences"] > df["subject"].map(fences)
    df["age_outlier"] = df["age"] >= AGE_OUTLIER
    df["inconsistent_grade"] = (df["G3"] > 0) & (df["G2"] == 0)
    log["absence_outlier_fence"] = {k: float(v) for k, v in fences.items()}
    log["rows_out"] = int(len(df))

    self.clean = df
    self.cleaning_log = log
    if self.pisa_df is not None:
        self.pisa_clean = clean_pisa(self.pisa_df)
    return df
```

Vì sao tin được: mỗi bước kiểm tra đều *đếm* số giá trị nó thay đổi và lưu vào `summary.json → cleaning_log`. Tổng cộng 0 lỗi — tức dữ liệu UCI gốc sạch về cấu trúc; vấn đề thật duy nhất là về ý nghĩa (G3 = 0, mục 6). Không có gì bị ép ngầm: giá trị lạ sẽ thành NaN và hiện ra trong các con số đếm này.

5.1 Cờ bất thường (gắn cờ, không xoá)

Cờ	Điều kiện	Số dòng (mat)	Số dòng (por)	Ý nghĩa
`dropout_suspect`	`G3 == 0`	38	15	Nghi bỏ thi / bỏ học
`g2_zero`	`G2 == 0`	13	7	Nghi vắng kỳ 2
`grade_drop`	`G1−G3 ≥ 8` hoặc `G2−G3 ≥ 6`, loại trừ dropout	0	1	Tụt điểm đột ngột
`absence_outlier`	`absences > Q3 + 1.5·IQR` (= 20 ở mat)	15	21	Vắng nhiều bất thường
`age_outlier`	`age ≥ 20`	5	9	Học muộn / lưu ban
`inconsistent_grade`	`G3 > 0` nhưng `G2 == 0`	0	0	Điểm mâu thuẫn

Ngưỡng lấy từ tuple `ANOMALY_THRESHOLDS = (1.5, 8, 6, 20)` (hệ số IQR, mức tụt G1, mức tụt G2, tuổi). Ngưỡng IQR tính riêng cho từng môn vì phân phối số buổi vắng của 2 môn khác nhau.

6. Nhóm nghi bỏ thi (G3 = 0)

Bằng chứng	Giá trị
Số học sinh có G3 = 0	38
... trong đó số buổi vắng = 0	**38 / 38**
... trong đó G2 = 0	13
G1 / G2 trung bình của nhóm	7.53 / 4.66
Tỉ lệ vắng = 0 ở các học sinh còn lại	21.6%
Số học sinh có G3 từ 1 đến 3	0 (khoảng trống ngay trước cột 0)

Diễn giải: các em này vẫn có điểm G1 thật, rồi điểm cuối kỳ bằng 0 và 0 buổi vắng được ghi nhận. Điểm 0/20 thật sau khi G1 ≈ 7.53 là khó tin; giải thích hợp lý là "không dự thi / nghỉ học, số buổi vắng không được ghi". Các em vẫn nằm trong `cleaned_students.csv` với `dropout_suspect = True`, và mọi thống kê được báo cáo 2 lần: **all** (n = 395) và **exclude_dropout** (n = 357).

Chỉ số	all (n=395)	exclude_dropout (n=357)
Tỉ lệ đỗ	67.1%	**74.2%**
r(G2, G3)	0.905	**0.966**
r(G1, G3)	0.801	**0.892**
r(vắng học, G3)	0.034	**-0.213**
r(số lần trượt, G3)	-0.36	**-0.294**

6.1 Phân tích độ nhạy: kết luận có phụ thuộc vào cách xử lý các số 0 không?

Đã làm gì. Có 2 giả định có thể sai: (1) G3 = 0 nghĩa là "bỏ thi", và (2) absences = 0 nghĩa là "không vắng buổi nào" — cũng có thể là "không được ghi nhận". Vì vậy phân tích vắng học được lặp lại theo 3 kịch bản, ở cả 2 môn, rồi so sánh kết quả.

Môn	Kịch bản	n	r(vắng, G3) [CI 95%]	p	G3 trung bình: vắng thấp / vừa / cao	ANOVA p	Tỉ lệ đỗ
mat	toàn bộ học sinh	395	**+0.034** [-0.065, 0.132]	0.497	9.82 / 11.38 / 10.37	0.013	67.1%
mat	bỏ nhóm nghi bỏ thi (G3 = 0)	357	**-0.213** [-0.31, -0.112]	< 0.001	12.26 / 11.38 / 10.37	< 0.001	74.2%
mat	bỏ thêm absences = 0 (có thể là 'không được ghi')	280	**-0.176** [-0.287, -0.06]	0.003	11.91 / 11.38 / 10.37	0.008	71.4%
por	toàn bộ học sinh	649	**-0.091** [-0.167, -0.015]	0.020	12.07 / 11.88 / 11.13	0.082	84.6%
por	bỏ nhóm nghi bỏ thi (G3 = 0)	634	**-0.196** [-0.27, -0.12]	< 0.001	12.57 / 11.88 / 11.13	< 0.001	86.6%
por	bỏ thêm absences = 0 (có thể là 'không được ghi')	405	**-0.142** [-0.237, -0.046]	0.004	12.12 / 11.88 / 11.13	0.034	83.7%

Mẫu hình bỏ thi ở cả 2 môn: mat: 38/38 em nghi bỏ thi có absences = 0 (so với 21.6% ở các em còn lại), por: 15/15 em nghi bỏ thi có absences = 0 (so với 36.1% ở các em còn lại).

Diễn giải:

- Ở mọi kịch bản đã bỏ các số 0, vắng học đều có quan hệ **âm và có ý nghĩa** với G3 ở cả 2 môn. Kết luận vẫn đúng dù giả định bỏ thi đúng hay sai, và dù absences = 0 là thật hay do không ghi nhận.
- Chỉ kịch bản "toàn bộ học sinh" làm mờ hoặc che mất ảnh hưởng — đúng là sự méo mó do các dòng G3 = 0 gây ra.
- Mẫu hình bỏ thi (G3 = 0 luôn đi kèm absences = 0) xuất hiện độc lập ở cả 2 môn, củng cố cách hiểu "không dự thi / nghỉ học" thay vì điểm thật.

`data_processor.py` — `StudentDataProcessor._sensitivity` (lines 752–787)

```python
def _sensitivity(self) -> dict:
    """Does the absences finding survive different assumptions about the zeros?

    - all: keep every student;
    - exclude_dropout: drop G3 = 0 (dropout suspects);
    - exclude_dropout_and_zero_absences: also drop absences = 0, in case 0 means
      'not recorded' rather than 'never absent'.
    """
    out: dict = {}
    for subj, g in self.clean.groupby("subject"):
        scenarios = {
            "all": g,
            "exclude_dropout": g[~g["dropout_suspect"]],
            "exclude_dropout_and_zero_absences": g[~g["dropout_suspect"] & (g["absences"] > 0)],
        }
        res = {}
        for name, d in scenarios.items():
            r = stats.pearsonr(d["absences"].astype(float), d["G3"].astype(float))
            ci = r.confidence_interval(1 - ALPHA)
            levels = [x["G3"].astype(float).to_numpy() for _, x in d.groupby("absence_level")]
            res[name] = {
                "n": int(len(d)),
                "r_absences_G3": round(float(r.statistic), 3), "p": _p(r.pvalue),
                "ci95": [round(float(ci.low), 3), round(float(ci.high), 3)],
                "mean_G3_by_absence_level": {
                    k: round(float(x["G3"].mean()), 2) for k, x in d.groupby("absence_level")},
                "anova_p": _p(stats.f_oneway(*levels).pvalue),
                "pass_rate": round(float((d["pass_fail"] == "pass").mean()), 4),
            }
        drop = g[g["dropout_suspect"]]
        res["dropout_pattern"] = {"n_dropout": int(len(drop)),
                                  "n_dropout_absences_zero": int((drop["absences"] == 0).sum()),
                                  "share_absences_zero_others": round(float(
                                      (g.loc[~g["dropout_suspect"], "absences"] == 0).mean()), 4)}
        out[subj] = res
    return out
```

7. Phân tích khám phá (groupby → mean ± std, n)

Mỗi ô là *G3 trung bình ± độ lệch chuẩn (n học sinh)*. In đậm = phạm vi dùng cho kết luận chính.

Theo thời gian tự học mỗi tuần

Studytime	all	exclude_dropout
1 (<2h)	10.05 ± 4.96 (n=105)	**11.47** ± 3.41 (n=92)
2 (2-5h)	10.17 ± 4.22 (n=198)	**11.07** ± 3.07 (n=182)
3 (5-10h)	11.4 ± 4.64 (n=65)	**12.56** ± 2.99 (n=59)
4 (>10h)	11.26 ± 5.28 (n=27)	**12.67** ± 3.6 (n=24)

Theo mức độ vắng — G3 trung bình giảm dần từ vắng thấp đến vắng cao khi đã bỏ nhóm nghi bỏ thi.

Mức vắng	all	exclude_dropout
thấp (low)	9.82 ± 5.61 (n=191)	**12.26** ± 3.04 (n=153)
trung bình (medium)	11.38 ± 3.25 (n=121)	**11.38** ± 3.25 (n=121)
cao (high)	10.37 ± 3.2 (n=83)	**10.37** ± 3.2 (n=83)

Theo số lần trượt trước đây

Số lần trượt	all	exclude_dropout
0	11.25 ± 4.17 (n=312)	**11.94** ± 3.19 (n=294)
1	8.12 ± 4.71 (n=50)	**10.15** ± 2.6 (n=40)
2	6.24 ± 4.84 (n=17)	**8.83** ± 3.01 (n=12)
3	5.69 ± 4.19 (n=16)	**8.27** ± 1.68 (n=11)

Theo internet ở nhà / hỗ trợ gia đình / hỗ trợ của trường / học thêm có trả phí

Nhóm	all	exclude_dropout
internet = không	9.41 ± 4.49 (n=66)	**10.71** ± 2.97 (n=58)
internet = có	10.62 ± 4.58 (n=329)	**11.68** ± 3.26 (n=299)
famsup = không	10.64 ± 4.64 (n=153)	**11.8** ± 3.18 (n=138)
famsup = có	10.27 ± 4.55 (n=242)	**11.35** ± 3.25 (n=219)
schoolsup = không	10.56 ± 4.77 (n=344)	**11.83** ± 3.22 (n=307)
schoolsup = có	9.43 ± 2.87 (n=51)	**9.62** ± 2.55 (n=50)
paid = không	9.99 ± 5.13 (n=214)	**11.61** ± 3.4 (n=184)
paid = có	10.92 ± 3.79 (n=181)	**11.43** ± 3.04 (n=173)

Lưu ý: học sinh `schoolsup = có` điểm thấp hơn vì trường *dành* hỗ trợ cho em học yếu (quan hệ nhân quả ngược) — đây không phải bằng chứng hỗ trợ làm giảm điểm.

`data_processor.py` — `_grp` (lines 408–413)

```
def _grp(df: pd.DataFrame, col: str) -> dict:
    g = df.groupby(col)["G3"].agg(["mean", "std", "count"])
    return {str(k): {"mean": round(float(v["mean"]), 2),
                     "std": round(float(v["std"]), 2) if pd.notna(v["std"]) else 0.0,
                     "n": int(v["count"])}
            for k, v in g.iterrows()}
```

`data_processor.py` — `_scope_stats` (lines 519–544)

```
def _scope_stats(df: pd.DataFrame) -> dict:
    corr = df[CORR_COLS].astype(float).corr().round(3)
    return {
        "n": int(len(df)),
        "pass_rate": round(float((df["pass_fail"] == "pass").mean()), 4),
        "n_pass": int((df["pass_fail"] == "pass").sum()),
        "n_fail": int((df["pass_fail"] == "fail").sum()),
        "by_studytime": _grp(df, "studytime"),
        "by_absence_level": _grp(df, "absence_level"),
        "by_failures": _grp(df, "failures"),
        "by_internet": _grp(df, "internet"),
        "by_studytime_internet": _grp(df.assign(st_net=df["studytime"].astype(str) + "|" + df["internet"]),
                                      "st_net"),
        "by_famsup": _grp(df, "famsup"),
        "by_schoolsup": _grp(df, "schoolsup"),
```

```
        "by_paid": _grp(df, "paid"),
        "by_risk_group": _grp(df, "risk_group"),
        "corr_G1_G2_G3": {
            "G1_G2": float(corr.loc["G1", "G2"]),
            "G1_G3": float(corr.loc["G1", "G3"]),
            "G2_G3": float(corr.loc["G2", "G3"]),
        },
        "corr_full": corr.to_dict(),
        "group_tests": _group_tests(df),
        "corr_tests": _corr_tests(df),
    }
```

7.1 Kiểm định thống kê: chênh lệch là thật hay do ngẫu nhiên?

Đã làm gì. Với mỗi so sánh ở trên, kiểm định thống kê trả lời: *nếu các nhóm thực sự có cùng G3 trung bình, thì khả năng thấy chênh lệch lớn cỡ này chỉ do ngẫu nhiên là bao nhiêu?* Xác suất đó là **p-value**; chênh lệch được gọi là có ý nghĩa thống kê khi $p < \alpha = 0.05$.

Phương pháp	Dùng cho	Vì sao chọn
Welch t-test	biến có/không (internet, famsup, schoolsup, paid)	So 2 trung bình mà không giả định phương sai bằng nhau (cỡ nhóm chênh nhiều, vd 58 vs 299)
ANOVA một chiều (F)	studytime (4 mức), mức vắng (3), số lần trượt (4)	So ≥ 3 trung bình trong một kiểm định
Kruskal–Wallis (H)	giống ANOVA	Kiểm tra lại dựa trên thứ hạng, không cần giả định điểm phân phối chuẩn
Hiệu chỉnh Holm–Bonferroni	cả 7 kiểm định cùng lúc	Làm nhiều kiểm định thì dễ ra "có ý nghĩa" giả; Holm điều chỉnh p để cả nhóm kiểm định giữ sai số 5%
Độ lớn hiệu ứng: Cohen's d / eta²	mọi kiểm định	p chỉ cho biết "thật hay không"; effect size cho biết "lớn cỡ nào". d: 0.2 nhỏ, 0.5 vừa, 0.8 lớn. eta² = tỉ lệ phương sai G3 được giải thích: 0.01 nhỏ, 0.06 vừa, 0.14 lớn
Khoảng tin cậy 95% của chênh lệch	t-test	Khoảng giá trị hợp lý của chênh lệch thật; nếu chứa 0 thì chênh lệch không có ý nghĩa

Kết quả (exclude_dropout, n = 357):

Biến	Kiểm định	Nhóm (G3 trung bình)	Thống kê	Chênh lệch [CI 95%] / kiểm tra lại	p	p (Holm)	Effect size	Có ý nghĩa?
`internet`	Welch t-test	có 11.68 (n=299) vs không 10.71 (n=58)	t = 2.25 (df 85.8)	+0.98 [0.11, 1.84]	0.027	0.080	Cohen's d = 0.3	❌ không
`famsup`	Welch t-test	có 11.35 (n=219) vs không 11.8 (n=138)	t = -1.28 (df 296.4)	-0.45 [-1.13, 0.24]	0.202	0.404	Cohen's d = -0.14	❌ không
`schoolsup`	Welch t-test	có 9.62 (n=50) vs không 11.83 (n=307)	t = -5.46 (df 76.9)	-2.21 [-3.02, -1.41]	< 0.001	< 0.001	Cohen's d = -0.71	✅ có
`paid`	Welch t-test	có 11.43 (n=173) vs không 11.61 (n=184)	t = -0.55 (df 354.1)	-0.19 [-0.86, 0.48]	0.585	0.585	Cohen's d = -0.06	❌ không
`studytime`	one-way ANOVA	4 nhóm	F = 4.38	Kruskal–Wallis H = 11.18, p 0.011	0.005	0.019	eta² = 0.036	✅ có

Biến	Kiểm định	Nhóm (G3 trung bình)	Thống kê	Chênh lệch [CI 95%] / kiểm tra lại	p	p (Holm)	Effect size	Có ý nghĩa?
`absence_level`	one-way ANOVA	3 nhóm	F = 9.85	Kruskal–Wallis H = 19.1, p < 0.001	< 0.001	< 0.001	eta^2 = 0.053	✅ có
`failures`	one-way ANOVA	4 nhóm	F = 11.49	Kruskal–Wallis H = 34.17, p < 0.001	< 0.001	< 0.001	eta^2 = 0.089	✅ có

Tương quan Pearson với G3 (p-value và CI 95% của r):

Biến	r	CI 95%	p
`G1`	+0.892	[0.868, 0.911]	< 0.001
`G2`	+0.966	[0.958, 0.972]	< 0.001
`studytime`	+0.127	[0.023, 0.228]	0.017
`failures`	-0.294	[-0.386, -0.196]	< 0.001
`absences`	-0.213	[-0.31, -0.112]	< 0.001
`famsup`	-0.067	[-0.17, 0.037]	0.205

Diễn giải:

- Có ý nghĩa sau hiệu chỉnh Holm: `schoolsup`, `studytime`, `absence_level`, `failures`. Không có ý nghĩa: `internet`, `famsup`, `paid`.
- Internet: chênh +0.98 điểm có p gốc = 0.027 nhưng p Holm = 0.080, nên **không còn ý nghĩa** sau hiệu chỉnh nhiều kiểm định. Chỉ nên trình bày như một chênh lệch yếu, chưa chắc chắn.
- Hỗ trợ gia đình: CI [-1.13, 0.24] chứa 0 → không có bằng chứng `famsup` làm thay đổi G3.
- Thời gian tự học có ý nghĩa nhưng rất nhỏ (eta^2 = 0.036: chỉ giải thích khoảng 3.6% phương sai của G3). Số lần trượt (eta^2 = 0.089) và vắng học (eta^2 = 0.053) quan trọng hơn.
- ANOVA và Kruskal–Wallis cho cùng kết luận ở mọi biến, nên kết luận không phụ thuộc giả định điểm phân phối chuẩn.

Vì sao tin được: kiểm định dùng `scipy.stats` (cài đặt chuẩn), dùng biến thể không cần phương sai bằng nhau và phi tham số ở chỗ giả định đáng ngờ, và đã hiệu chỉnh cho nhiều kiểm định.

`data_processor.py` — `_group_tests` (lines 433–473)

```
def _group_tests(df: pd.DataFrame) -> dict:
    """Is the G3 difference between groups larger than chance?

    - yes/no columns: Welch t-test (unequal variances), 95% CI of the mean
      difference, Cohen's d effect size.
    - multi-level columns: one-way ANOVA with eta-squared effect size, plus
      Kruskal-Wallis as a check that does not assume normal data.
    All p-values of the family are Holm-adjusted before comparing to ALPHA.
    """
    tests: dict = {}
    for col in BINARY_TEST_COLS:
        yes = df.loc[df[col] == "yes", "G3"].astype(float)
        no = df.loc[df[col] == "no", "G3"].astype(float)
        res = stats.ttest_ind(yes, no, equal_var=False)
        ci = res.confidence_interval(1 - ALPHA)
        pooled_sd = np.sqrt(((len(yes) - 1) * yes.var() + (len(no) - 1) * no.var())
                            / (len(yes) + len(no) - 2))
        diff = yes.mean() - no.mean()
        tests[col] = {"test": "Welch t-test", "n_yes": int(len(yes)), "n_no": int(len(no)),
```

```
                    "mean_yes": round(float(yes.mean()), 2), "mean_no": round(float(no.mean()), 2),
                    "diff": round(float(diff), 2),
                    "ci95": [round(float(ci.low), 2), round(float(ci.high), 2)],
                    "stat": round(float(res.statistic), 2), "df": round(float(res.df), 1),
                    "p": _p(res.pvalue), "effect": round(float(diff / pooled_sd), 2),
                    "effect_name": "Cohen's d"}
    for col in MULTI_TEST_COLS:
        groups = [g["G3"].astype(float).to_numpy() for _, g in df.groupby(col)]
        anova = stats.f_oneway(*groups)
        kw = stats.kruskal(*groups)
        grand = df["G3"].mean()
        ss_between = sum(len(g) * (g.mean() - grand) ** 2 for g in groups)
        ss_total = float(((df["G3"] - grand) ** 2).sum())
        tests[col] = {"test": "one-way ANOVA", "k_groups": len(groups),
                      "stat": round(float(anova.statistic), 2), "p": _p(anova.pvalue),
                      "effect": round(float(ss_between / ss_total), 3), "effect_name": "eta²",
                      "kruskal_H": round(float(kw.statistic), 2), "p_kruskal": _p(kw.pvalue)}
    adjusted = _holm({k: v["p"] for k, v in tests.items()})
    for k, v in tests.items():
        v["p_holm"] = _p(adjusted[k])
        v["significant"] = bool(adjusted[k] < ALPHA)
    return tests
```

`data_processor.py` — `_holm` (lines 424–430)

```
def _holm(pvals: dict[str, float]) -> dict[str, float]:
    """Holm-Bonferroni adjusted p-values (controls the family-wise error rate)."""
    out, running, m = {}, 0.0, len(pvals)
    for i, (k, p) in enumerate(sorted(pvals.items(), key=lambda kv: kv[1])):
        running = max(running, min(1.0, (m - i) * p))
        out[k] = running
    return out
```

`data_processor.py` — `_corr_tests` (lines 476–484)

```
def _corr_tests(df: pd.DataFrame) -> dict:
    """Pearson r with G3: p-value and 95% CI for each variable."""
    out = {}
    for col in CORR_TEST_COLS:
        res = stats.pearsonr(df[col].astype(float), df["G3"].astype(float))
        ci = res.confidence_interval(1 - ALPHA)
        out[col] = {"r": round(float(res.statistic), 3), "p": _p(res.pvalue),
                    "ci95": [round(float(ci.low), 3), round(float(ci.high), 3)]}
    return out
```

7.2 Hồi quy đa biến: yếu tố nào vẫn quan trọng khi giữ các yếu tố khác cố định?

Đã làm gì. So sánh từng biến riêng lẻ (7.1) không phân biệt được một yếu tố tự nó có tác dụng hay chỉ đi kèm yếu tố khác (vd internet ↔ bố mẹ học vấn cao). **Hồi quy tuyến tính OLS** ước lượng tất cả yếu tố cùng lúc: mỗi hệ số **b** là mức thay đổi kỳ vọng của G3 (điểm) khi biến đó tăng 1 đơn vị **trong khi mọi biến khác giữ nguyên**. Sai số chuẩn dùng HC3 (không cần giả định phương sai bằng nhau). Chỉ dùng học sinh có G3 > 0. Hai mô hình:

- **Mô hình A — yếu tố nền** (13 biến: các biến của câu hỏi chính + học vấn bố/mẹ, giới tính, tuổi, nơi ở).
- **Mô hình B — A + G1, G2**: còn yếu tố nào quan trọng khi đã biết điểm giữa kỳ.

Mô hình	mat: n / R^2 / R^2 hiệu chỉnh	por: n / R^2 / R^2 hiệu chỉnh
A: yếu tố nền	357 / 0.234 / 0.205	634 / 0.321 / 0.307
B: A + G1, G2	357 / 0.937 / 0.934	634 / 0.886 / 0.884

R^2 = tỉ lệ biến động của G3 mà mô hình giải thích được (0 = không gì, 1 = toàn bộ).

Hệ số mô hình A (mat; cột cuối chạy lại cùng mô hình trên por để kiểm tra lặp lại):

Biến	b (điểm G3)	CI 95%	p	Có ý nghĩa (mat)	por: b, có ý nghĩa
studytime (+1 mức)	+0.514	[0.081, 0.948]	0.020	✅	+0.409 ✅
failures (+1 lần trượt)	-0.898	[-1.385, -0.411]	< 0.001	✅	-1.290 ✅
absences (+1 buổi vắng)	-0.064	[-0.106, -0.023]	0.003	✅	-0.079 ✅
famsup (có vs không)	-0.520	[-1.214, 0.174]	0.142	—	-0.239 —
schoolsup (có vs không)	-2.184	[-3.11, -1.258]	< 0.001	✅	-0.962 ✅
học thêm trả phí (có vs không)	-0.603	[-1.278, 0.071]	0.079	—	-0.647 —
internet (có vs không)	+0.527	[-0.367, 1.421]	0.247	—	+0.289 —
muốn học đại học (có vs không)	+0.203	[-1.399, 1.805]	0.803	—	+1.833 ✅
học vấn mẹ (+1 bậc)	+0.267	[-0.1, 0.635]	0.154	—	+0.391 ✅
học vấn bố (+1 bậc)	+0.186	[-0.194, 0.566]	0.336	—	+0.071 —
giới tính (nam vs nữ)	+0.402	[-0.255, 1.06]	0.230	—	-0.582 ✅
tuổi (+1 năm)	-0.210	[-0.49, 0.071]	0.143	—	+0.247 ✅
nơi ở (thành thị vs nông thôn)	+0.604	[-0.179, 1.388]	0.130	—	+0.315 —

Mô hình B, chỉ phần điểm (toàn bộ hệ số có trong `summary.json → regression`):

Biến	b (điểm G3)	CI 95%	p	Có ý nghĩa (mat)	por: b, có ý nghĩa
G1 (+1 điểm)	+0.120	[0.051, 0.188]	< 0.001	✅	+0.192 ✅
G2 (+1 điểm)	+0.870	[0.802, 0.938]	< 0.001	✅	+0.746 ✅

Diễn giải:

- Có ý nghĩa ở **cả 2 môn** khi giữ mọi yếu tố khác cố định: studytime (+1 mức), failures (+1 lần trượt), absences (+1 buổi vắng), schoolsup (có vs không).
- Internet: chênh lệch thô là +0.98; khi giữ các yếu tố khác cố định b = +0.53 (p 0.247) → **không có tác động riêng**.
- Hỗ trợ gia đình (`famsup`): b = -0.52 (p 0.142) → không đo được tác động.
- Các yếu tố nền chỉ giải thích R^2 = 0.234 (mat): phần lớn khác biệt giữa học sinh *không* được giải thích bởi các biến này.
- Khi đã biết G1, G2 (mô hình B, R^2 = 0.937), chỉ còn absences (+1 buổi vắng) giữ tác động riêng có ý nghĩa ở mat: điểm giữa kỳ đã chứa gần như toàn bộ thông tin.
- Hồi quy chỉ điều chỉnh cho các biến có trong dữ liệu; kết quả là quan hệ tương quan, không phải nhân quả.

`data_processor.py` — `_ols` (lines 487–515)

```
def _ols(df: pd.DataFrame, cols: list[str]) -> dict:
    """Linear regression of G3 on `cols` with HC3 heteroskedasticity-robust standard errors.

    Each coefficient b = expected change in G3 (points) for +1 unit of that column while
    every other column stays fixed. `male` and `urban` are 0/1 versions of sex / address.
    """
    X = df.assign(male=(df["sex"] == "M").astype(int),
                  urban=(df["address"] == "U").astype(int))[cols].astype(float).to_numpy()
    y = df["G3"].astype(float).to_numpy()
    A = np.column_stack([np.ones(len(y)), X])
    n, k = A.shape
    xtx_inv = np.linalg.inv(A.T @ A)
    beta = xtx_inv @ A.T @ y
    resid = y - A @ beta
```

```
    leverage = np.einsum("ij,jk,ik->i", A, xtx_inv, A)
    meat = A.T @ (A * (resid ** 2 / (1 - leverage) ** 2)[:, None])
    se = np.sqrt(np.diag(xtx_inv @ meat @ xtx_inv))
    dof = n - k
    p = 2 * stats.t.sf(np.abs(beta / se), dof)
    q = stats.t.ppf(1 - ALPHA / 2, dof)
    r2 = 1 - (resid @ resid) / ((y - y.mean()) @ (y - y.mean()))
    return {
        "n": int(n), "n_predictors": int(k - 1),
        "r2": round(float(r2), 3), "adj_r2": round(float(1 - (1 - r2) * (n - 1) / dof), 3),
        "coef": {name: {"b": round(float(b), 3), "se": round(float(s), 3),
                        "ci95": [round(float(b - q * s), 3), round(float(b + q * s), 3)],
                        "p": _p(pv), "significant": bool(pv < ALPHA)}
                 for name, b, s, pv in zip(["intercept"] + cols, beta, se, p)},
    }
```

`data_processor.py` — `StudentDataProcessor._regression` (lines 739–750)

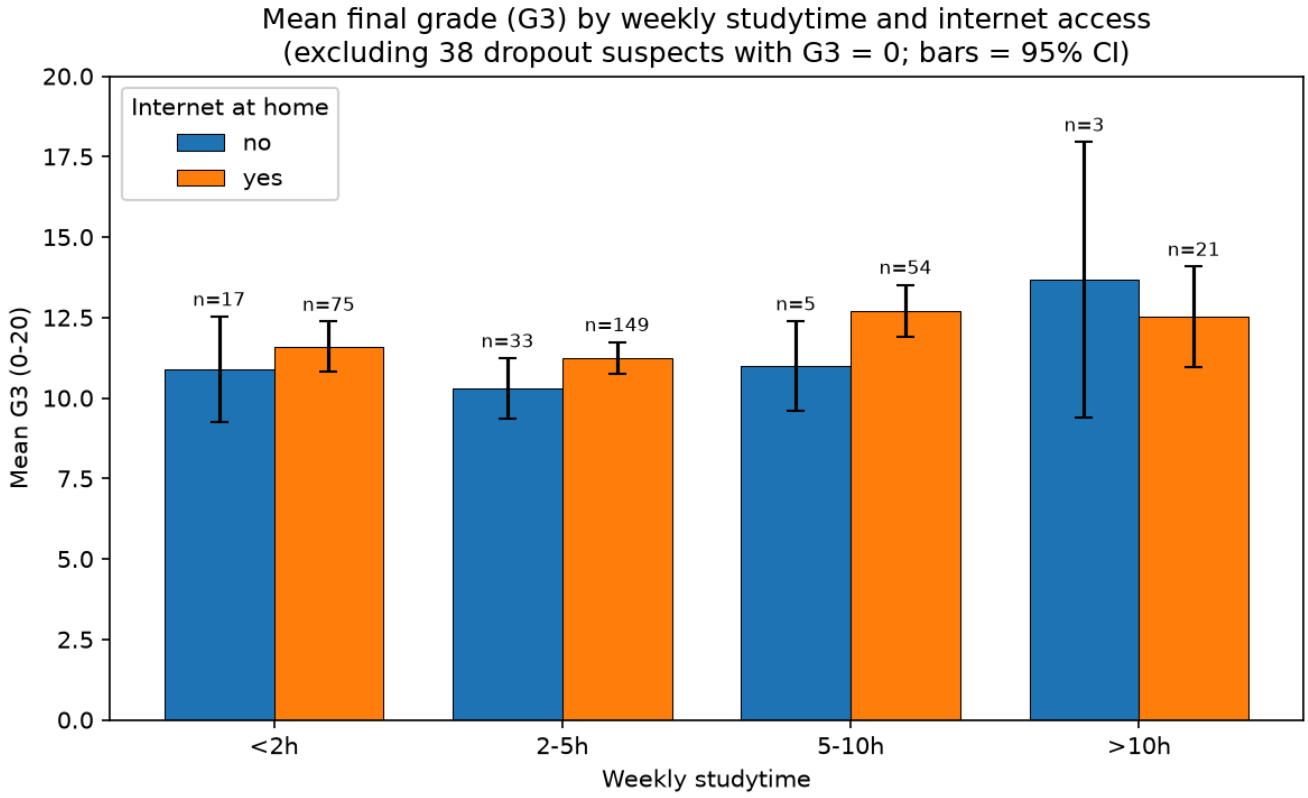
```
def _regression(self) -> dict:
    """Multivariable OLS per subject (dropout suspects excluded).

    factors: background variables only (what predicts G3 before any grade is known);
    factors_plus_grades: the same plus G1, G2 (what still matters once grades are known).
    """
    out: dict = {}
    for subj, g in self.clean.groupby("subject"):
        core = g[~g["dropout_suspect"]]
        out[subj] = {"factors": _ols(core, REG_FACTORS),
                     "factors_plus_grades": _ols(core, REG_FACTORS + REG_GRADES)}
    return out
```

8. Biểu đồ

Ba ảnh đầu do pipeline sinh ra (`analyze()` → `_plot_charts()`), không vẽ tay. Xoá thư mục `images/` rồi chạy `python main.py` là ảnh được tạo lại. Ảnh thứ tư (bối cảnh PISA) nằm ở mục 10.2.

8.1 Biểu đồ 1 — Biểu đồ cột nhóm: G3 trung bình theo thời gian tự học và internet



Định nghĩa. Biểu đồ cột nhóm (grouped bar chart) so sánh *giá trị trung bình* của một biến số (G3) giữa các tổ hợp của hai biến phân loại (mức thời gian tự học × có internet ở nhà hay không). Dữ liệu: 357 học sinh (đã bỏ nhóm nghi bỏ thi).

Thành phần	Ý nghĩa
Trục X	Mức thời gian tự học mỗi tuần: 1 = <2h, 2 = 2–5h, 3 = 5–10h, 4 = >10h
Trục Y	Điểm cuối kỳ G3 trung bình, thang 0–20 (cố định 0–20 để so chiều cao cột cho công bằng)
Cột xanh dương	Học sinh **không có** internet ở nhà
Cột cam	Học sinh **có** internet ở nhà
Thanh đen (error bar)	Khoảng tin cậy 95% của trung bình: mean ± 1.96·std/√n. Thanh càng dài thì trung bình càng kém chắc chắn
Nhãn `n=`	Số học sinh trong cột đó
Chú giải (legend)	Màu → nhóm internet

Cách đọc: so chiều cao cột *cùng với* thanh sai số; nếu 2 thanh chồng lên nhau nhiều thì chênh lệch không đáng tin. Các nhóm rất ít học sinh: studytime 4 & internet=không (n=3), studytime 3 & internet=không (n=5).

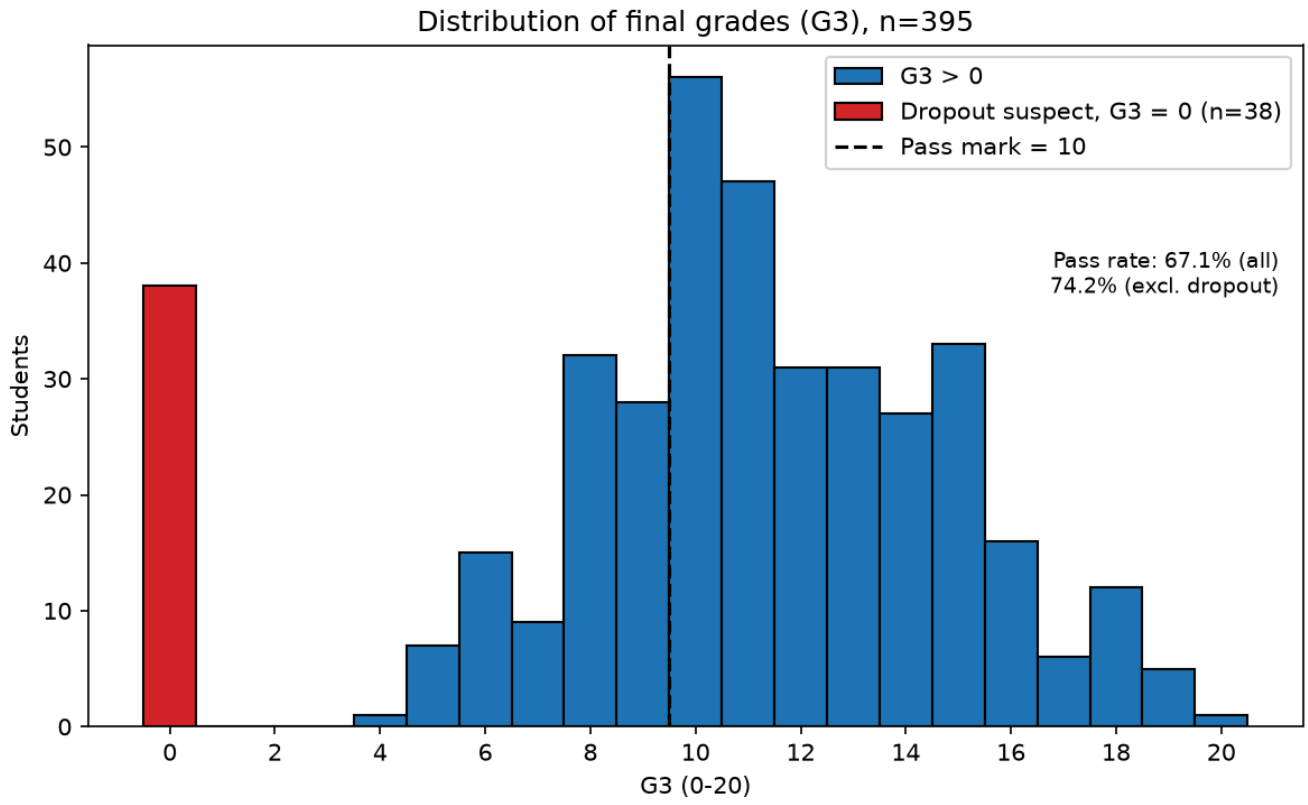
Kết luận: có internet 11.68 vs không có 10.71, nhưng chênh lệch này **không có ý nghĩa** sau hiệu chỉnh Holm (mục 7.1); các thanh sai số chồng lên nhau trên biểu đồ cũng cho thấy điều đó. Thời gian tự học chỉ có xu hướng tăng nhẹ (r = 0.127).

`data_processor.py` — `StudentDataProcessor._plot_bar` (lines 848–877)

```
def _plot_bar(self, df: pd.DataFrame, core: pd.DataFrame) -> None:
    """Chart 1: grouped bar - mean G3 by studytime x internet, 95% CI + n."""
    n_drop = int(df["dropout_suspect"].sum())
```

```
    stats = core.groupby(["studytime", "internet"])["G3"].agg(["mean", "std", "count"])
    stats["ci"] = 1.96 * stats["std"].fillna(0) / np.sqrt(stats["count"])
    levels = sorted(core["studytime"].unique())
    fig, ax = plt.subplots(figsize=(8, 5))
    width = 0.38
    for k, net in enumerate(["no", "yes"]):
        xs, means, cis, ns = [], [], [], []
        for i, lvl in enumerate(levels):
            if (lvl, net) in stats.index:
                s = stats.loc[(lvl, net)]
                xs.append(i + (k - 0.5) * width)
                means.append(s["mean"]); cis.append(s["ci"]); ns.append(int(s["count"]))
        bars = ax.bar(xs, means, width, yerr=cis, capsize=4, label=net,
                      color=f"C{k}", edgecolor="black", linewidth=0.5)
        for b, n, ci in zip(bars, ns, cis):
            ax.text(b.get_x() + b.get_width() / 2, b.get_height() + ci + 0.2,
                    f"n={n}", ha="center", va="bottom", fontsize=8)
    ax.set_xticks(range(len(levels)), ["<2h", "2-5h", "5-10h", ">10h"], rotation=0)
    ax.set_title(f"Mean final grade (G3) by weekly studytime and internet access\n"
                 f"(excluding {n_drop} dropout suspects with G3 = 0; bars = 95% CI)")
    ax.set_xlabel("Weekly studytime")
    ax.set_ylabel("Mean G3 (0-20)")
    ax.set_ylim(0, 20)
    ax.legend(title="Internet at home", loc="upper left")
    fig.tight_layout()
    fig.savefig(self.images_dir / "01_bar_studytime_internet.png", dpi=150)
    plt.close(fig)
```

8.2 Biểu đồ 2 — Histogram điểm cuối kỳ (G3)



Định nghĩa. Histogram chia một biến số thành các khoảng (bin) và cho biết có bao nhiêu quan sát rơi vào mỗi khoảng. Ở đây mỗi bin đúng bằng 1 điểm (tâm bin tại 0, 1, …, 20). Dữ liệu: toàn bộ 395 học sinh.

Thành phần	Ý nghĩa
Trục X	Điểm cuối kỳ G3 (0–20)
Trục Y	Số học sinh đạt điểm đó

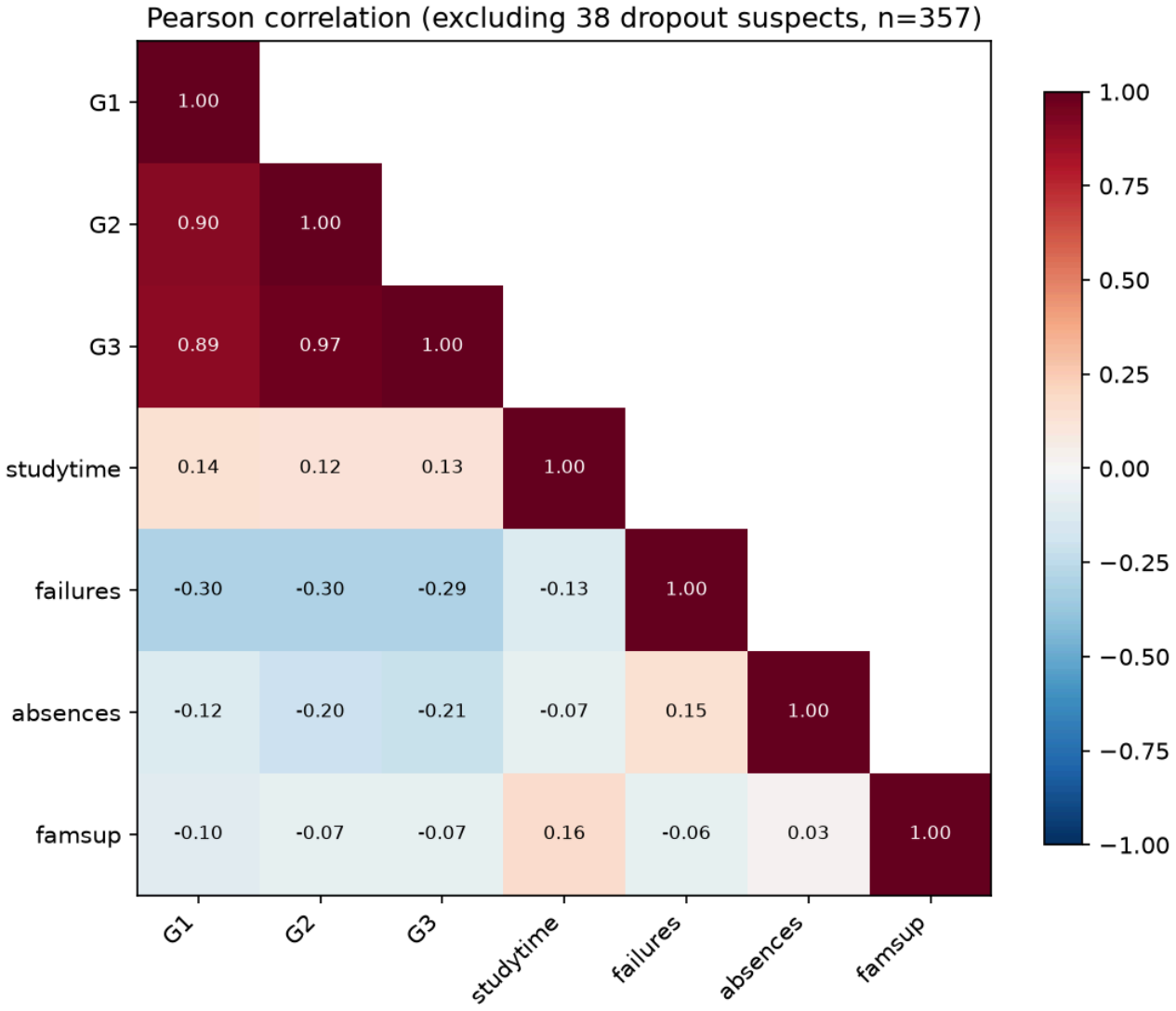
Thành phần	Ý nghĩa
Cột xanh dương	Học sinh có G3 > 0
Cột đỏ tại 0	Nhóm nghỉ bỏ thi (G3 = 0, n = 38)
Đường đứt nét dọc	Mốc đỗ: bên trái = trượt (G3 < 10), bên phải = đỗ (G3 ≥ 10)
Ô chữ	Tỉ lệ đỗ trên toàn bộ học sinh và sau khi bỏ nhóm nghỉ bỏ thi
Chú giải (legend)	Ý nghĩa màu / đường

Kết luận: điểm thật phân bố gần hình chuông quanh 10 (điểm xuất hiện nhiều nhất), còn cột 0 tách hẳn ra với khoảng trống ở 1–3 điểm (0 học sinh) — bằng chứng trực quan rằng điểm 0 là hiện tượng khác, không phải đuôi thấp của phân phối điểm.

`data_processor.py` — `StudentDataProcessor._plot_hist` (lines 879–901)

```
def _plot_hist(self, df: pd.DataFrame, core: pd.DataFrame) -> None:
    """Chart 2: histogram - G3 distribution, one bin per integer grade."""
    n_drop = int(df["dropout_suspect"].sum())
    fig, ax = plt.subplots(figsize=(8, 5))
    bins = np.arange(-0.5, 21.5, 1)
    ax.hist(core["G3"], bins=bins, edgecolor="black", color="C0", label="G3 > 0")
    ax.hist(df.loc[df["dropout_suspect"], "G3"], bins=bins, edgecolor="black",
            color="C3", label=f"Dropout suspect, G3 = 0 (n={n_drop})")
    ax.axvline(PASS_MARK - 0.5, color="black", linestyle="--",
               label=f"Pass mark = {PASS_MARK}")
    pass_all = (df["pass_fail"] == "pass").mean()
    pass_core = (core["pass_fail"] == "pass").mean()
    ax.text(0.98, 0.70, f"Pass rate: {pass_all:.1%} (all)\n"
                        f"{pass_core:.1%} (excl. dropout)",
            transform=ax.transAxes, ha="right", va="top", fontsize=9)
    ax.set_xticks(range(0, 21, 2))
    ax.set_title(f"Distribution of final grades (G3), n={len(df)}")
    ax.set_xlabel("G3 (0-20)")
    ax.set_ylabel("Students")
    ax.legend()
    fig.tight_layout()
    fig.savefig(self.images_dir / "02_hist_g3.png", dpi=150)
    plt.close(fig)
```

8.3 Biểu đồ 3 — Heatmap tương quan



Định nghĩa. Heatmap biểu diễn một ma trận số bằng màu sắc. Mỗi ô là **hệ số tương quan Pearson** r giữa hai biến: r = cov(X, Y) / (σX·σY), nằm trong khoảng −1 (tương quan tuyến tính âm hoàn toàn), 0 (không có quan hệ tuyến tính) đến +1 (dương hoàn toàn). Dữ liệu: 357 học sinh (đã bỏ nhóm nghi bỏ thi).

Thành phần	Ý nghĩa
Hàng / cột	Các biến: G1, G2, G3 (điểm), studytime (1–4), failures (0–3), absences (số buổi vắng), famsup (1 = có, 0 = không)
Màu ô	Đỏ = r dương, xanh = r âm, trắng ≈ 0; màu càng đậm thì quan hệ càng mạnh
Số trong ô	Giá trị r (2 chữ số thập phân)
Đường chéo (1.00)	Mỗi biến với chính nó — luôn bằng 1
Nửa trên để trống	Ẩn đi vì ma trận đối xứng (r(X,Y) = r(Y,X))
Thanh màu	Quy đổi màu sang giá trị r từ −1 đến +1

Ghi chú: `famsup` là biến 0/1 nên r của nó là tương quan point-biserial (cùng công thức).

Kết luận — tương quan với G3, từ mạnh đến yếu: G2 +0.966, G1 +0.892, failures -0.294, absences -0.213, studytime +0.127, famsup -0.067.

`data_processor.py` — `StudentDataProcessor._plot_heatmap` (lines 903–922)

```python
def _plot_heatmap(self, df: pd.DataFrame, core: pd.DataFrame) -> None:
    """Chart 3: heatmap - Pearson correlation (excluding dropout), diverging colormap."""
    n_drop = int(df["dropout_suspect"].sum())
    corr = core[CORR_COLS].astype(float).corr()
    labels = [c.replace("_bin", "") for c in CORR_COLS]
    fig, ax = plt.subplots(figsize=(7.5, 6.5))
    mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
    im = ax.imshow(np.ma.array(corr.values, mask=mask), cmap="RdBu_r", vmin=-1, vmax=1)
    ax.set_xticks(range(len(labels)), labels, rotation=45, ha="right")
    ax.set_yticks(range(len(labels)), labels)
    for i in range(len(labels)):
        for j in range(i + 1):
            v = corr.values[i, j]
            ax.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=8,
                    color="white" if abs(v) > 0.6 else "black")
    ax.set_title(f"Pearson correlation (excluding {n_drop} dropout suspects, n={len(core)})")
    fig.colorbar(im, ax=ax, shrink=0.8)
    fig.tight_layout()
    fig.savefig(self.images_dir / "03_heatmap_corr.png", dpi=150)
    plt.close(fig)
```

9. Rule cảnh báo sớm nguy cơ trượt

Đã làm gì: `classify_risk()` **chỉ dùng thông tin có trước kỳ thi cuối** (G1, G2, số lần trượt, số buổi vắng — không bao giờ dùng G3), nên có thể đánh giá trung thực với kết quả đỗ/trượt thật.

- high (cao): G2 < 10 hoặc số lần trượt ≥ 2
- medium (trung bình): G1 < 10 hoặc số lần trượt ≥ 1 hoặc vắng ≥ 10
- low (thấp): còn lại

`data_processor.py` — `classify_risk` (lines 137–151)

```python
def classify_risk(row: pd.Series) -> str:
    """Early-warning segment from information available before the final exam.

    Uses only G1, G2, failures, absences (never G3, so it can be evaluated
    against pass/fail without leakage).
    high   : G2 < G_WARN or failures >= FAIL_HIGH
    medium : G1 < G_WARN or failures > 0 or absences high (>= ABS_HIGH)
    low    : otherwise
    """
    if row["G2"] < G_WARN or row["failures"] >= FAIL_HIGH:
        return "high"
    if (row["G1"] < G_WARN or row["failures"] > 0
            or classify_absences(row["absences"]) == "high"):
        return "medium"
    return "low"
```

Đánh giá trên toàn bộ 395 học sinh (cảnh báo = nhóm "high", thực tế = G3 < 10):

	trượt thật	**đỗ thật**
bị cảnh báo high	122	27
không bị cảnh báo	8	238

- **Recall** = TP / (TP + FN) = 122 / 130 = **93.8%** số em trượt thật được cảnh báo sớm (CI 95% 88.3%–96.9%).

- **Precision** = TP / (TP + FP) = 122 / 149 = **81.9%** số em bị cảnh báo thực sự trượt (CI 95% 74.9%–87.2%).
- Tỉ lệ trượt theo nhóm: low 1.2%, medium 7.8%, high 81.9%.

`data_processor.py` — `_risk_eval` (lines 555–574)

```
def _risk_eval(df: pd.DataFrame) -> dict:
    """Confusion counts of risk flags vs. actual fail (G3 < PASS_MARK), with 95% Wilson CIs."""
    actual = df["pass_fail"] == "fail"
    out = {"n": int(len(df)), "fail_rate": round(float(actual.mean()), 4),
           "fail_rate_by_group": {
               g: round(float(actual[df["risk_group"] == g].mean()), 4)
               for g in ["low", "medium", "high"] if (df["risk_group"] == g).any()},
           "tiers": _tier_eval(df, actual)}
    for name, flagged in {"high": df["risk_group"] == "high",
                          "high_or_medium": df["risk_group"] != "low"}.items():
        tp = int((flagged & actual).sum())
        fp = int((flagged & ~actual).sum())
        fn = int((~flagged & actual).sum())
        tn = int((~flagged & ~actual).sum())
        out[name] = {"tp": tp, "fp": fp, "fn": fn, "tn": tn,
                     "precision": round(tp / (tp + fp), 4) if tp + fp else 0.0,
                     "precision_ci95": _wilson(tp, tp + fp),
                     "recall": round(tp / (tp + fn), 4) if tp + fn else 0.0,
                     "recall_ci95": _wilson(tp, tp + fn)}
    return out
```

Kiểm tra ngoài mẫu (holdout). Các ngưỡng ở trên được chọn khi đã nhìn dữ liệu môn Toán, nên các con số trên là *in-sample* và có thể lạc quan. Để kiểm tra, **giữ nguyên rule** (không đổi ngưỡng nào) rồi áp lên môn Bồ Đào Nha, dữ liệu mà rule chưa từng thấy. Vì nhiều học sinh học cả 2 môn, bản holdout chặt chỉ giữ những học sinh môn Bồ Đào Nha có 13 cột nhận dạng (`MERGE_KEYS`, giống `student-merge.R`) không xuất hiện trong file môn Toán. Khoảng tin cậy là khoảng Wilson 95%.

Tập dữ liệu	n	Tỉ lệ trượt	TP / FP / FN	Recall [CI 95%]	Precision [CI 95%]
mat — in-sample (rule được đặt trên dữ liệu này)	395	32.9%	122 / 27 / 8	**93.8%** [88.3%, 96.9%]	**81.9%** [74.9%, 87.2%]
por — toàn bộ học sinh (holdout)	649	15.4%	90 / 62 / 10	**90.0%** [82.6%, 94.5%]	**59.2%** [51.3%, 66.7%]
por — chỉ học sinh không có trong mat (holdout chặt)	275	25.4%	64 / 33 / 6	**91.4%** [82.5%, 96.0%]	**66.0%** [56.1%, 74.6%]

Diễn giải:

- **Recall** trên học sinh chưa thấy: 91.4% so với 93.8% in-sample — rule vẫn bắt được 64/70 em sẽ trượt.
- **Precision giảm** (66.0% so với 81.9%): báo động nhầm nhiều hơn. Một phần vì môn Bồ Đào Nha ít em trượt hơn (tỉ lệ trượt 25.4% so với 32.9%); precision luôn giảm khi kết quả cần bắt hiếm hơn.
- Kết luận: rule là công cụ *sàng lọc* tốt (ít bỏ sót em trượt), không phải dự đoán chính xác; con số trung thực nên trích dẫn là con số holdout.

`data_processor.py` — `_wilson` (lines 547–552)

```
def _wilson(k: int, n: int) -> list[float]:
    """95% Wilson confidence interval of a proportion k/n."""
    if n == 0:
        return [0.0, 0.0]
    ci = stats.binomtest(k, n).proportion_ci(1 - ALPHA, method="wilson")
    return [round(float(ci.low), 4), round(float(ci.high), 4)]
```

`data_processor.py` — `StudentDataProcessor._risk_holdout` (lines 789–805)

```python
def _risk_holdout(self) -> dict:
    """Evaluate the frozen risk rule on the other subject(s) (out-of-sample).

    The rule thresholds were chosen on the primary subject, so other
    subjects act as holdout data. `holdout_<s>_only` keeps only students
    whose MERGE_KEYS do not appear in the primary subject (no shared pupils).
    """
    primary = self.clean[self.clean["subject"] == self.primary_subject]
    seen = set(map(tuple, primary[MERGE_KEYS].astype(str).to_numpy()))
    out: dict = {}
    for subj, g in self.clean.groupby("subject"):
        if subj == self.primary_subject:
            continue
        shared = g[MERGE_KEYS].astype(str).apply(tuple, axis=1).isin(seen)
        out[f"holdout_{subj}"] = _risk_eval(g)
        out[f"holdout_{subj}_only"] = _risk_eval(g[~shared])
    return out
```

Chia 3 nhóm (low / medium / high) có hợp lý không? Bảng cho biết bao nhiêu em trong mỗi nhóm thực sự trượt (CI Wilson 95%), và nhóm "medium" có thực sự rủi ro hơn "low" không (nguy cơ tương đối, kiểm định chính xác Fisher — phù hợp khi số đếm nhỏ).

Tập dữ liệu	low: trượt / n	medium: trượt / n	high: trượt / n	medium so với low
mat (dữ liệu đặt rule)	2/169 = **1.2%** [0.3%, 4.2%]	6/77 = **7.8%** [3.6%, 16.0%]	122/149 = **81.9%** [74.9%, 87.2%]	×6.6 (p 0.013) ✅
por holdout chặt (chưa thấy)	1/127 = **0.8%** [0.1%, 4.3%]	5/51 = **9.8%** [4.3%, 21.0%]	64/97 = **66.0%** [56.1%, 74.6%]	×12.4 (p 0.008) ✅

Nhóm medium trượt nhiều gấp nhiều lần nhóm low, cả trên dữ liệu dùng để đặt rule lẫn trên học sinh chưa từng thấy (kiểm định Fisher có ý nghĩa ở cả 2), trong khi vẫn thấp xa nhóm high. Vì vậy 3 nhóm đều mang thông tin và ứng với 3 hành động: **high → can thiệp ngay**, **medium → theo dõi**, **low → không cần can thiệp**. Precision/recall chính dùng nhóm high làm cảnh báo.

`data_processor.py` — `_tier_eval` (lines 577–593)

```python
def _tier_eval(df: pd.DataFrame, actual: pd.Series) -> dict:
    """Does each risk tier carry information? Fail rate (Wilson CI) per tier, and
    Fisher's exact test of medium vs low (is 'medium' really riskier than 'low'?)."""
    out: dict = {}
    for g in ["low", "medium", "high"]:
        m = df["risk_group"] == g
        k, n = int((actual & m).sum()), int(m.sum())
        out[g] = {"n": n, "n_fail": k, "fail_rate": round(k / n, 4) if n else 0.0,
                  "fail_rate_ci95": _wilson(k, n)}
    lo, me = out["low"], out["medium"]
    table = [[me["n_fail"], me["n"] - me["n_fail"]], [lo["n_fail"], lo["n"] - lo["n_fail"]]]
    fisher = stats.fisher_exact(table)
    out["medium_vs_low"] = {
        "odds_ratio": round(float(fisher.statistic), 2) if np.isfinite(fisher.statistic) else None,
        "relative_risk": round(me["fail_rate"] / lo["fail_rate"], 1) if lo["fail_rate"] else None,
        "p": _p(fisher.pvalue), "significant": bool(fisher.pvalue < ALPHA)}
    return out
```

Vì sao tin được: không rò rỉ dữ liệu (G3 không phải đầu vào — thấy rõ trong `classify_risk()`), độ bất định được thể hiện bằng khoảng tin cậy, và hiệu năng được kiểm chứng trên học sinh mà rule chưa từng được chỉnh theo.

10. Bối cảnh

10.1 Môn Toán vs môn Bồ Đào Nha (kiểm tra lặp lại)

Chỉ số	mat	por
n	395	649
n_dropout_suspect	38	15
pass_rate	0.6709	0.8459
mean_G3	10.42	11.91
mean_G3_exclude_dropout	11.52	12.19
corr_G2_G3_exclude_dropout	0.966	0.934
corr_absences_G3_exclude_dropout	-0.213	-0.196
corr_failures_G3_exclude_dropout	-0.294	-0.388

Chiều của các tương quan chính (G2, vắng học, số lần trượt với G3) **giống nhau** ở cả 2 môn, nên kết luận chính lặp lại được trên một file dữ liệu khác. Lưu ý: nhiều học sinh có mặt ở cả 2 môn.

`data_processor.py` — `StudentDataProcessor._subject_comparison` (lines 807–821)

```
def _subject_comparison(self) -> dict:
    out = {}
    for subj, g in self.clean.groupby("subject"):
        core = g[~g["dropout_suspect"]]
        out[subj] = {
            "n": int(len(g)),
            "n_dropout_suspect": int(g["dropout_suspect"].sum()),
            "pass_rate": round(float((g["pass_fail"] == "pass").mean()), 4),
            "mean_G3": round(float(g["G3"].mean()), 2),
            "mean_G3_exclude_dropout": round(float(core["G3"].mean()), 2),
            "corr_G2_G3_exclude_dropout": round(float(core["G2"].corr(core["G3"])), 3),
            "corr_absences_G3_exclude_dropout": round(float(core["absences"].corr(core["G3"])), 3),
            "corr_failures_G3_exclude_dropout": round(float(core["failures"].corr(core["G3"])), 3),
        }
    return out
```

10.2 PISA: Bồ Đào Nha so với trung bình OECD

Học sinh trong dữ liệu UCI đến từ 2 trường ở Bồ Đào Nha (năm học 2005–2006, Cortez & Silva 2008). PISA chỉ cung cấp bối cảnh **cấp quốc gia**; không bao giờ ghép với từng dòng học sinh.

Môn	Năm	Bồ Đào Nha	TB OECD	Chênh lệch	Hạng Bồ Đào Nha
mathematics	2000	454	492	-38	23
mathematics	2003	466	499	-33	30
mathematics	2006	466	494	-28	35
mathematics	2009	487	495	-8	31
mathematics	2012	487	494	-7	29
mathematics	2015	492	490	+2	29
mathematics	2018	492	489	+3	28
mathematics	2022	472	472	+0	29

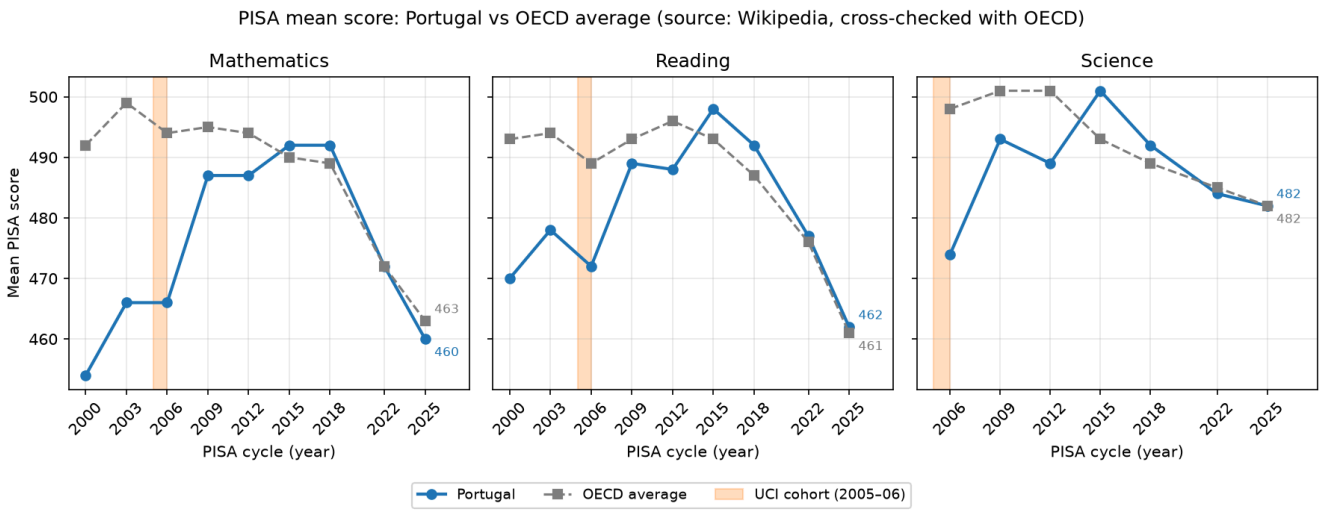
Môn	Năm	Bồ Đào Nha	TB OECD	Chênh lệch	Hạng Bồ Đào Nha
mathematics	2025	460	463	-3	30
reading	2000	470	493	-23	26
reading	2003	478	494	-16	28
reading	2006	472	489	-17	30
reading	2009	489	493	-4	25
reading	2012	488	496	-8	31
reading	2015	498	493	+5	21
reading	2018	492	487	+5	24
reading	2022	477	476	+1	24
reading	2025	462	461	+1	26
science	2006	474	498	-24	36
science	2009	493	501	-8	30
science	2012	489	501	-12	34
science	2015	501	493	+8	23
science	2018	492	489	+3	26
science	2022	484	485	-1	30
science	2025	482	482	+0	29

Môn Toán: Bồ Đào Nha 454 điểm năm 2000 → 460 điểm năm 2025 (OECD 463).

`data_processor.py` — `StudentDataProcessor._pisa_context` (lines 823–838)

```
def _pisa_context(self) -> dict:
    """Portugal vs OECD average per subject/year (country-level context only)."""
    p = self.pisa_clean
    sel = p[p["country"].isin(["Portugal", "OECD average"])]
    out: dict = {}
    for (subject, year), g in sel.groupby(["subject", "year"]):
        by = g.set_index("country")
        if "Portugal" not in by.index:
            continue
        row = {"portugal": int(by.loc["Portugal", "score"]),
               "portugal_rank": (int(by.loc["Portugal", "rank"])
                                 if pd.notna(by.loc["Portugal", "rank"]) else None)}
        if "OECD average" in by.index:
            row["oecd_average"] = int(by.loc["OECD average", "score"])
        out.setdefault(subject, {})[str(year)] = row
    return out
```

Biểu đồ 4 — Xu hướng PISA: Bồ Đào Nha so với trung bình OECD



Định nghĩa. Biểu đồ đường (line chart) cho thấy một giá trị thay đổi theo thời gian. Mỗi ô là một môn PISA; mỗi điểm là điểm PISA trung bình của cả nước trong một kỳ thi (thang PISA được chuẩn hoá sao cho trung bình OECD khoảng 500 ở lần đầu môn đó là môn chính). Dữ liệu: `pisa_clean.csv` (1662 dòng), đã đối chiếu với OECD ở mục 3.4.

Thành phần	Ý nghĩa
Các ô	Toán, Đọc hiểu, Khoa học (bảng so sánh có Khoa học từ 2006)
Trục X	Năm của kỳ thi PISA (2000, 2003, ..., 2022, 2025)
Trục Y	Điểm PISA trung bình (dùng chung cho 3 ô để so mức điểm)
Đường xanh liền, chấm tròn	Bồ Đào Nha
Đường xám đứt nét, ô vuông	Trung bình OECD
Số ở cuối mỗi đường	Điểm mới nhất (2025)
Dải màu cam	Năm học 2005–06, lúc khảo sát học sinh trong dữ liệu UCI

Cách đọc: khoảng cách dọc giữa 2 đường là mức chênh của Bồ Đào Nha so với trung bình OECD trong năm đó.

Kết luận: môn Toán, Bồ Đào Nha đi từ 454 điểm (2000, OECD 492) đến 460 điểm (2025, OECD 463). Lần đầu bắt kịp trung bình OECD vào năm 2015 và từ đó đi cùng nhịp với OECD. Vào thời điểm dữ liệu UCI (dải cam, kỳ gần nhất là 2006), Bồ Đào Nha thấp hơn trung bình OECD ở cả 3 môn.

`data_processor.py` — `StudentDataProcessor._plot_pisa` (lines 924–953)

```
def _plot_pisa(self) -> None:
    """Chart 4 (context): Portugal vs OECD average PISA score per subject over time."""
    p = self.pisa_clean
    fig, axes = plt.subplots(1, 3, figsize=(12, 4.6), sharey=True)
    styles = {"Portugal": dict(color="C0", marker="o", lw=2),
              "OECD average": dict(color="gray", marker="s", ls="--", lw=1.5)}
    for ax, subject in zip(axes, ["mathematics", "reading", "science"]):
        series = {c: p[(p["subject"] == subject) & (p["country"] == c)].sort_values("year")
                  for c in styles}
        last = {c: s.iloc[-1] for c, s in series.items()}
        higher = max(last, key=lambda c: last[c]["score"])
        for country, s in series.items():
            ax.plot(s["year"], s["score"], label=country, **styles[country])
            ax.annotate(f"{last[country]['score']}", (last[country]["year"], last[country]["score"]),
                        textcoords="offset points", xytext=(6, 5 if country == higher else -11),
                        fontsize=8, color=styles[country]["color"])
```

```
        ax.axvspan(2005, 2006, color="C1", alpha=0.25,
                   label="UCI cohort (2005-06)" if subject == "mathematics" else None)
        ax.set_title(subject.capitalize())
        ax.set_xlabel("PISA cycle (year)")
        ax.set_xticks(sorted(p.loc[p["subject"] == subject, "year"].unique()))
        ax.tick_params(axis="x", rotation=45)
        ax.set_xlim(right=ax.get_xlim()[1] + 2)
        ax.grid(alpha=0.3)
    axes[0].set_ylabel("Mean PISA score")
    fig.legend(*axes[0].get_legend_handles_labels(), loc="lower center", ncol=3, fontsize=9)
    fig.suptitle("PISA mean score: Portugal vs OECD average (source: Wikipedia, cross-checked with OECD)")
    fig.tight_layout(rect=(0, 0.07, 1, 1))
    fig.savefig(self.images_dir / "04_pisa_portugal_oecd.png", dpi=150)
    plt.close(fig)
```

11. Pipeline (OOP) và đầu ra

`main.py` chạy: `fetch_uci_raw` → `scrape_pisa_table` (cache-first) → `load_data` → `clean_data` → `analyze` (+ vẽ biểu đồ) → `export_results` → sinh báo cáo này.

`main.py` (whole file)

```
"""P1 entry point: collect raw data, then run the full StudentDataProcessor pipeline."""
import argparse
from pathlib import Path

import matplotlib

matplotlib.use("Agg")  # script run: write PNG files only, never open plot windows

from data_processor import StudentDataProcessor, fetch_uci_raw, read_pisa_meta, scrape_pisa_table  # noqa: E402
from report_builder import build_reports  # noqa: E402

ROOT = Path(__file__).resolve().parent
RAW = ROOT / "data" / "raw"
RAW_PATHS = {"mat": RAW / "student-mat.csv", "por": RAW / "student-por.csv"}
PISA_RAW = RAW / "pisa_reference.csv"
OECD_RAW = RAW / "oecd_pisa2025_table_I1.csv"
PROCESSED = ROOT / "data" / "processed"
IMAGES = ROOT / "images"

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--refresh-pisa", action="store_true",
                        help="re-scrape Wikipedia PISA instead of using the cached raw CSV")
    args = parser.parse_args()

    fetch_uci_raw(RAW)

    # Reference scrape: cache-first, live only when missing or --refresh-pisa
    try:
        pisa = scrape_pisa_table(out_csv=PISA_RAW, refresh=args.refresh_pisa)
        meta = read_pisa_meta(PISA_RAW) or {}
        print(f"PISA raw: {len(pisa)} rows <- {PISA_RAW.name} "
              f"(revision {meta.get('revision_id')}, scraped {meta.get('scraped_at')})")
    except Exception as exc:  # noqa: BLE001 - reference data is optional
        print(f"PISA scrape skipped ({type(exc).__name__}: {exc})")

    proc = StudentDataProcessor(RAW_PATHS, PISA_RAW, PROCESSED, IMAGES, primary_subject="mat",
                                oecd_raw=OECD_RAW)
    df = proc.load_data()
    print(f"Loaded raw: {len(df)} rows ({df['subject'].value_counts().to_dict()})")
    clean = proc.clean_data()
    print(f"Cleaned: {len(clean)} rows, log={ {k: v for k, v in proc.cleaning_log.items() if isinstance(v, int)}
}")

    res = proc.analyze()
    for scope, s in res["scopes"].items():
        corr = s["corr_full"]
        print(f"[mat/{scope}] n={s['n']} pass_rate={s['pass_rate']} "
```

```
            f"G2-G3={s['corr_G1_G2_G3']['G2_G3']} absences-G3={corr['G3']['absences']}")
    high = res["risk_eval"]["all"]["high"]
    print(f"Risk rule (high vs fail): precision={high['precision']} recall={high['recall']}")
    for key in [k for k in res["risk_eval"] if k.startswith("holdout_")]:
        h = res["risk_eval"][key]["high"]
        print(f"  {key}: n={res['risk_eval'][key]['n']} precision={h['precision']} recall={h['recall']}")
    tests = res["scopes"]["exclude_dropout"]["group_tests"]
    print("Significant after Holm: " + ", ".join(f"{k} (p_holm={v['p_holm']})"
                                                  for k, v in tests.items() if v["significant"]))
    print("Not significant: " + ", ".join(f"{k} (p_holm={v['p_holm']})"
                                           for k, v in tests.items() if not v["significant"]))
    print(f"Anomaly counts: {res['anomaly_counts']}")
    check = res.get("pisa_oecd_check", {})
    for year, c in ((k, v) for k, v in check.items() if k.isdigit()):
        print(f"PISA vs OECD {year}: {c['n_within_tolerance']}/{c['n_compared']} within ±{c['tolerance']}, "
              f"exact {c['n_exact']}, mismatches {c['mismatches']}")

    cleaned_csv, summary_json = proc.export_results()
    print(f"Wrote {cleaned_csv}")
    print(f"Wrote {summary_json}")
    print("Charts: 01_bar_studytime_internet.png, 02_hist_g3.png, 03_heatmap_corr.png")
    for path in build_reports(proc, ROOT):
        print(f"Wrote {path}")

if __name__ == "__main__":
    main()
```

`data_processor.py` — `StudentDataProcessor.analyze` (lines 697–737)

```
def analyze(self) -> dict:
    assert self.clean is not None, "call clean_data() first"
    df = self.clean[self.clean["subject"] == self.primary_subject]
    core = df[~df["dropout_suspect"]]

    self.results = {
        "primary_subject": self.primary_subject,
        # set() uniques (rubric)
        "unique_schools": sorted(set(df["school"])),
        "unique_mjobs": sorted(set(df["Mjob"])),
        "unique_fjobs": sorted(set(df["Fjob"])),
        "thresholds": list(THRESHOLDS),
        "anomaly_thresholds": list(ANOMALY_THRESHOLDS),
        "cleaning_log": self.cleaning_log,
        "anomaly_counts": {
            subj: {f: int(g[f].sum()) for f in FLAG_COLS}
            for subj, g in self.clean.groupby("subject")},
        "scopes": {"all": _scope_stats(df), "exclude_dropout": _scope_stats(core)},
        "risk_eval": {"all": _risk_eval(df), "exclude_dropout": _risk_eval(core),
                      **self._risk_holdout()},
        "subject_comparison": self._subject_comparison(),
        "regression": self._regression(),
        "sensitivity": self._sensitivity(),
        "dropout_profile": {
            "n": int(df["dropout_suspect"].sum()),
            "n_absences_zero": int((df.loc[df["dropout_suspect"], "absences"] == 0).sum()),
            "n_g2_zero": int((df.loc[df["dropout_suspect"], "G2"] == 0).sum()),
            "mean_G1": round(float(df.loc[df["dropout_suspect"], "G1"].mean()), 2),
            "mean_G2": round(float(df.loc[df["dropout_suspect"], "G2"].mean()), 2),
            "share_absences_zero_others": round(float(
                (df.loc[~df["dropout_suspect"], "absences"] == 0).mean()), 4),
        },
    }
    if self.pisa_clean is not None:
        self.results["pisa_context"] = self._pisa_context()
        self.results["pisa_source"] = read_pisa_meta(self.pisa_raw)
        if self.oecd_df is not None:
            self.results["pisa_oecd_check"] = cross_check_pisa(self.pisa_clean, self.oecd_df)
            self.results["pisa_oecd_source"] = read_pisa_meta(self.oecd_raw)
    self._plot_charts(df, core)
    return self.results
```

`data_processor.py` — `StudentDataProcessor.export_results` (lines 956–972)

```
def export_results(self) -> tuple[Path, Path]:
    assert self.clean is not None and self.results, "run clean+analyze first"
    self.processed_dir.mkdir(parents=True, exist_ok=True)
    cleaned_csv = self.processed_dir / "cleaned_students.csv"
    summary_json = self.processed_dir / "summary.json"
    try:
        self.clean.to_csv(cleaned_csv, index=False)
        if self.pisa_clean is not None:
            self.pisa_clean.to_csv(self.processed_dir / "pisa_clean.csv", index=False)
    except PermissionError as exc:
        raise PermissionError(
            f"Không thể ghi vào '{self.processed_dir}' vì file đang được mở trong Excel/WPS. "
            f"Hãy đóng file đó rồi chạy lại."
        ) from exc
    with open(summary_json, "w", encoding="utf-8") as f:
        json.dump(self.results, f, ensure_ascii=False, indent=2)
    return cleaned_csv, summary_json
```

Đầu ra	Nội dung
`data/processed/cleaned_students.csv`	1044 dòng (cả 2 môn, cột `subject`), cột đã encode, cột tạo thêm, 6 cờ bất thường
`data/processed/pisa_clean.csv`	1662 dòng PISA đã làm sạch
`data/processed/summary.json`	toàn bộ thống kê trong báo cáo này
`images/*.png`	3 biểu đồ theo đề + 1 biểu đồ bối cảnh PISA
`report.md`, `report_vi.md`	báo cáo này (tiếng Anh / tiếng Việt)

12. Vì sao kết quả đáng tin (tổng hợp kiểm chứng)

Kiểm tra	Kết quả
Dữ liệu thô giống bản tải từ UCI	Tải từ URL chính thức; 395 / 649 dòng, 33 cột đúng như tài liệu
Các bước làm sạch (kiểu, miền giá trị, yes/no, trùng, thiếu)	Đều được đếm trong `cleaning_log`; tổng số lỗi = 0
Xử lý rõ ràng G3 = 0	Gắn cờ; 38/38 em có 0 buổi vắng; kết quả trình bày cả có và không có nhóm này
Độ nhạy	So sánh kết luận chính giữa `all` và `exclude_dropout` (mục 6)
Lặp lại trên dữ liệu khác	Cùng chiều tương quan ở môn Bồ Đào Nha: có
Rule rủi ro không rò rỉ	G3 không phải đầu vào của `classify_risk()`
Kiểm định thống kê	Welch t / ANOVA + Kruskal–Wallis, hiệu chỉnh Holm; có ý nghĩa: schoolsup, studytime, absence_level, failures; không có ý nghĩa: internet, famsup, paid
Hồi quy đa biến	OLS với sai số HC3; mô hình A R^2 = 0.234; có ý nghĩa ở cả 2 môn: studytime, failures, absences, schoolsup_bin
Độ nhạy với các số 0	Ảnh hưởng của vắng học âm và có ý nghĩa ở mọi kịch bản đã bỏ số 0, cả 2 môn: có
Các nhóm rủi ro	Nguy cơ trượt medium so với low ×6.6 (mat), ×12.4 (por chưa thấy); có ý nghĩa ở cả 2: có
Rule rủi ro ngoài mẫu	Rule cố định trên học sinh chỉ học môn Bồ Đào Nha: recall 91.4%, precision 66.0%
Cào PISA	Kiểm tra cấu trúc + cố định revision Wikipedia 1378067005
PISA so với OECD chính thức	2025: 274/274 điểm trùng khớp; 2022 (suy ra): 208/211 lệch không quá ±1
Biểu đồ	Sinh bởi code ở mục 8; tái tạo bằng `python main.py`
Số liệu trong báo cáo	Sinh từ kết quả pipeline bởi `report_builder.py`; không thể bị cũ

13. Giới hạn và hướng tiếp theo

- Chỉ 1 môn / 1 năm học làm nhóm chính; các nhóm nhỏ (xem nhãn `n=` ở Biểu đồ 1) có khoảng tin cậy rộng.
- Hồi quy chỉ điều chỉnh cho các biến có trong dữ liệu; các yếu tố không đo được (động lực, giáo viên, thu nhập) vẫn có thể gây nhiễu, nên đây là quan hệ tương quan, không phải nhân quả.
- "Nghi bỏ thi" là suy luận từ dữ liệu, chưa được trường xác nhận; phân tích độ nhạy (6.1) cho thấy kết luận về vắng học không phụ thuộc vào giả định này.
- Tập holdout cùng trường, cùng năm học với dữ liệu gốc, và holdout chặt khá nhỏ (n = 275); precision ở trường khác có thể khác.
- Điểm PISA 2025 và 2022 đã đối chiếu với báo cáo OECD; các kỳ cũ hơn (2000–2018) chỉ dựa vào Wikipedia.
- Mọi kết luận là tương quan, không phải nhân quả.

Tái tạo: `pip install -r requirements.txt` *rồi* `python main.py` *(thêm* `--refresh-pisa` *để cào lại Wikipedia).*